

Cuestiones de diseño e implementación

A menudo, se invoca con frecuencia al método *getInstancia* del Singleton. En aplicaciones con varios hilos de ejecución, la etapa de creación de la lógica de **inicialización perezosa** (*lazy*) es una sección crítica que requiere el control de concurrencia del hilo. Por tanto, asumiendo que la instancia se inicializa de manera perezosa, es habitual envolver el método con control de concurrencia. En Java, por ejemplo:

```
public static synchronized FactoriaDeServicios getInstancia()
{
    if ( instancia == null )
    {
        //sección crítica si es una aplicación con varios hilos
        instancia = new FactoriaDeServicios();
    }
    return instancia;
}
```

A propósito de la inicialización perezosa, ¿no es preferible una **inicialización impaciente** (*eager*), como en este ejemplo?

```
public class FactoriaDeServicios
{
    //inicialización impaciente
    private static FactoriaDeServicios instancia =
        new FactoriaDeServicios();

    public static FactoriaDeServicios getInstancia()
    {
        return instancia;
    }

    //otros métodos...
}
```

Es preferible el primer enfoque de inicialización perezosa al menos por estas razones:

- Se evita el trabajo de creación (y quizás retener recursos “caros”) si nunca se accede a la instancia.
- La inicialización perezosa de *getInstancia* algunas veces contiene lógica de creación compleja y condicional.

Otra pregunta típica de la implementación del Singleton es: ¿por qué no hacer que todos los métodos de los servicios sean métodos *estáticos* de la propia clase, en lugar de utilizar un objeto instancia con métodos de instancia? Por ejemplo, qué pasa si añadimos un método *estático* denominado *getAdaptadorContabilidad* a la *FactoriaDeServicios*. Pero normalmente es preferible utilizar una instancia y los métodos de instancia por estos motivos:

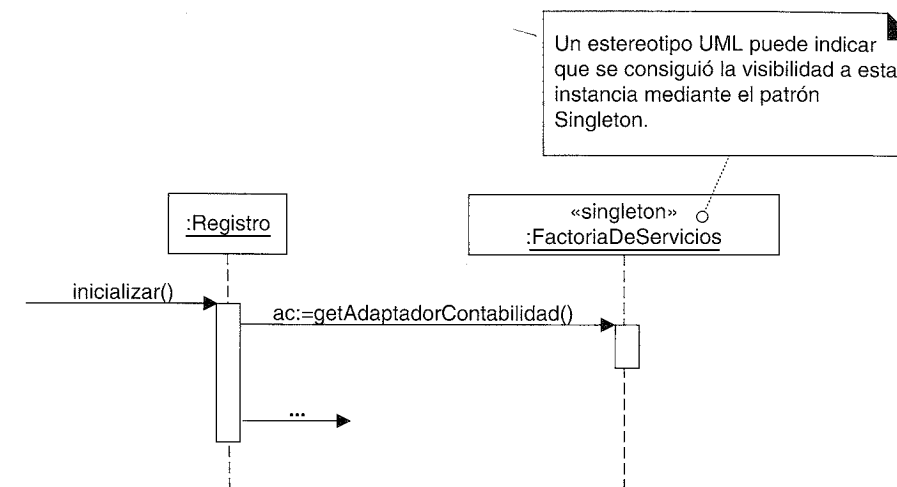


Figura 23.6. Mensaje implícito del patrón Singleton *getInstancia* que se indica en UML con un estereotipo.

- Los métodos de instancia permiten que se definan subclases y se refinan la clase singleton; los métodos estáticos no son polimórficos (virtual) y en la mayoría de los lenguajes (excepto Smalltalk) no se pueden redefinir en las subclases.
- La mayoría de los mecanismos de comunicación remota orientada a objetos (por ejemplo el RMI de Java) sólo soportan la activación remota de los métodos de instancia, no de los métodos estáticos. Se podría activar de manera remota una instancia singleton, aunque es verdad que raramente se hace.
- Una clase no siempre es un singleton en todos los contextos de aplicación. En la aplicación X, podría ser un singleton, pero podría ser una “multi-tonelada” en la aplicación Y. También es habitual comenzar un diseño pensando que será un singleton y luego descubrir que se necesitan múltiples instancias en el mismo proceso. Por tanto, la solución basada en la instancia es más flexible.

Patrones Relacionados

El patrón Singleton se utiliza a menudo para los objetos Factoría y Fachada —otro patrón GoF que se presentará—.

23.5. Conclusiones del problema de los servicios externos con diversas interfaces

Se ha utilizado una combinación de los patrones Adaptador, Factoría y Singleton para proporcionar Variaciones Protegidas contra las diversas interfaces de los calculadores de impuestos externos, sistemas de contabilidad, etcétera. La Figura 23.7 ilustra un contexto más amplio de la utilización de los patrones en la realización del caso de uso.

Este diseño podría no ser ideal, y siempre se puede mejorar. Pero uno de los objetivos que intenta conseguir este caso de estudio es demostrar que se puede construir por lo menos un diseño a partir de un conjunto de principios o patrones que son los “componentes básicos”, y que existe un enfoque metódico para llevar a cabo y explicar un di-

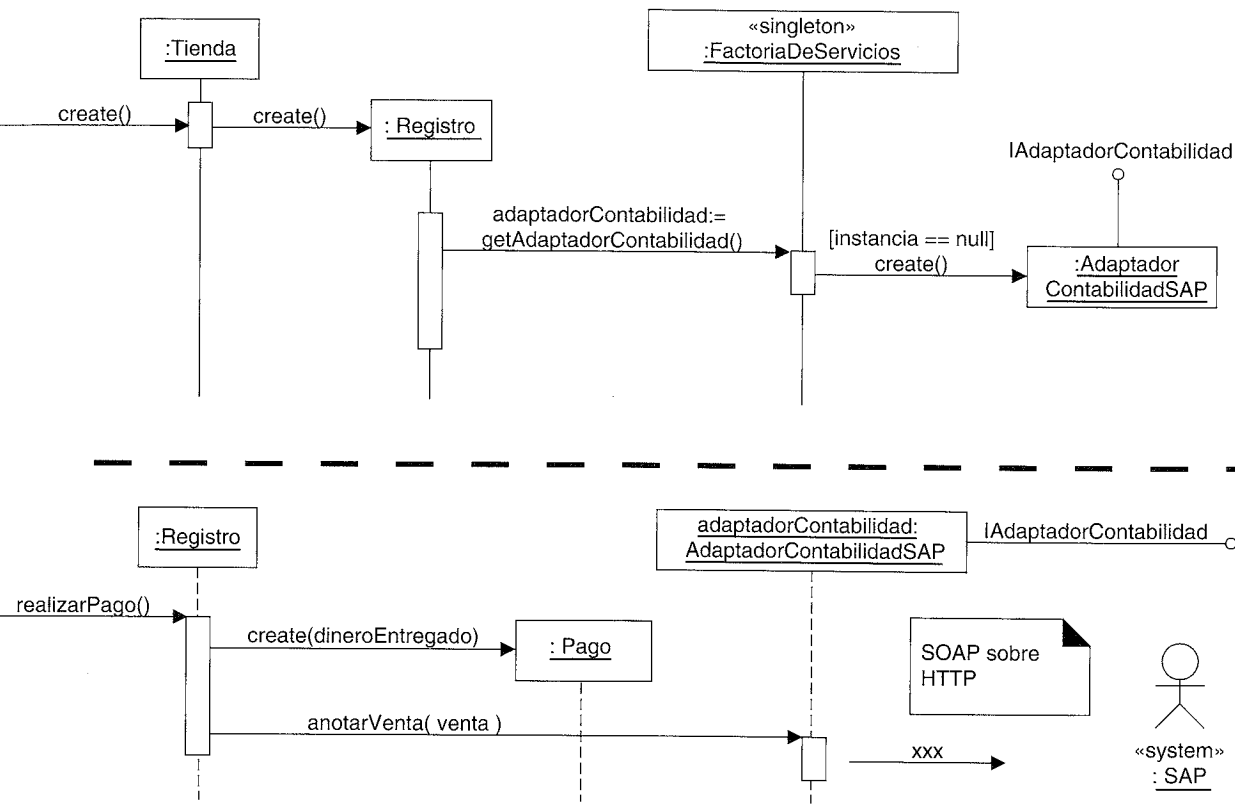


Figura 23.7. Patrones Adaptador, Factoría y Singleton aplicados al diseño.

seño. Espero sinceramente que se pueda ver cómo surge el diseño de la Figura 23.7 razonando en base al Controlador, Creador, Variaciones Protegidas, Bajo Acoplamiento, Alta Cohesión, Indirección, Polimorfismo, Adaptador, Factoría y Singleton.

Nótese lo conciso que puede ser un diseñador en una conversación o en la documentación cuando hay un conocimiento común de los patrones. Puedo decir, “Para resolver el problema de tener diversas interfaces para los servicios externos, podemos utilizar Adaptadores generados desde una Factoría Singleton”. En realidad, los diseñadores de objetos tienen conversaciones de este tipo; utilizando los patrones y los nombres de los patrones se eleva el nivel de abstracción en la comunicación del diseño.

23.6. Estrategia (GoF)

El siguiente problema de diseño que se va a resolver es proporcionar una lógica más compleja para fijar los precios, como descuentos para un día en toda la tienda, descuentos para las personas mayores, etcétera.

La estrategia de fijación de precios (que podría llamarse también regla, política o algoritmo) de una venta puede variar. Durante un periodo podría ser el 10% de descuento en todas las ventas, después podría ser un descuento de 10 € si el total de la venta es superior a 200 €, y muchas otras variaciones. ¿Cómo diseñamos los diversos algoritmos de fijación de precios?

Estrategia (Strategy)

Contexto/Problema

¿Cómo diseñar diversos algoritmos o políticas que están relacionadas? ¿Cómo diseñar que estos algoritmos o políticas puedan cambiar?

Solución

Defina cada algoritmo/política/estrategia en una clase independiente, con una interfaz común.

Puesto que el comportamiento de la fijación de precios varía según la estrategia (o algoritmo), creamos múltiples clases *EstrategiaFijarPreciosVenta*, cada una con un método polimórfico *getTotal* (ver Figura 23.8). A cada método *getTotal* se le pasa como parámetro el objeto *Venta*, de manera que el objeto de la estrategia de fijación de precios puede encontrar el precio anterior al descuento de la *Venta*, para aplicar después la regla de descuento. La implementación de cada *getTotal* será diferente: la *EstrategiaFijarPreciosPorcentajeDescuento*, aplicará el descuento de acuerdo a un porcentaje, y así sucesivamente.

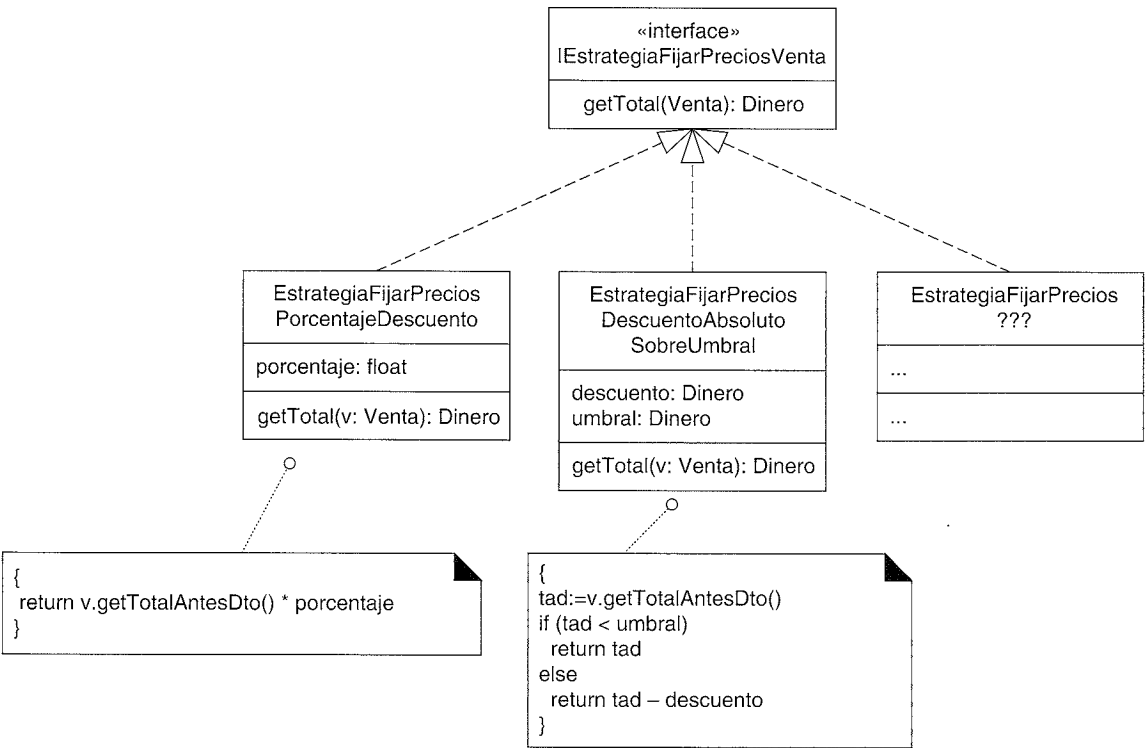


Figura 23.8. Clases para la Estrategia de fijación de precios.

Un objeto estrategia se conecta a un **objeto de contexto** —el objeto al que se aplica el algoritmo—. En este ejemplo, el objeto de contexto es una *Venta*. Cuando se envía el mensaje *getTotal* a la *Venta*, delega parte del trabajo a su objeto estrategia, como se ilustra en la Figura 23.9. No es necesario que el mensaje que se envía al objeto de contexto

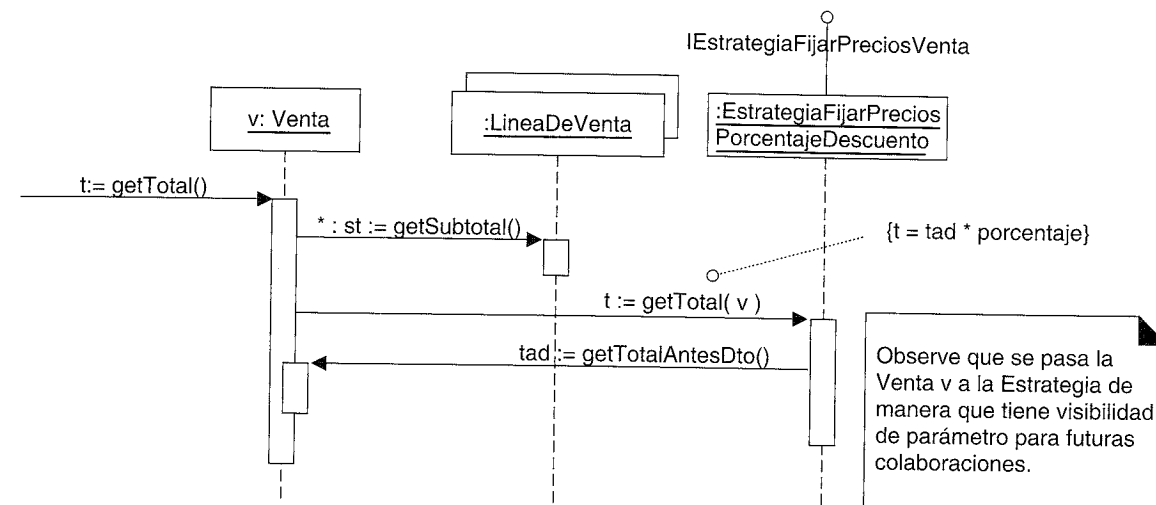


Figura 23.9. Estrategia en colaboración.

y al objeto estrategia tengan el mismo nombre, como en este ejemplo (por ejemplo, *getTotal* y *getTotal*), aunque es lo normal. Sin embargo, es habitual —de hecho, normalmente necesario— que el objeto de contexto pase una referencia a él mismo (*this*) al objeto estrategia, de manera que la estrategia tenga visibilidad de parámetro del objeto contexto, para futuras colaboraciones.

Obsérvese que el objeto de contexto (*Venta*) necesita tener visibilidad de atributo de su estrategia. Esto se refleja en el DCD de la Figura 23.10.

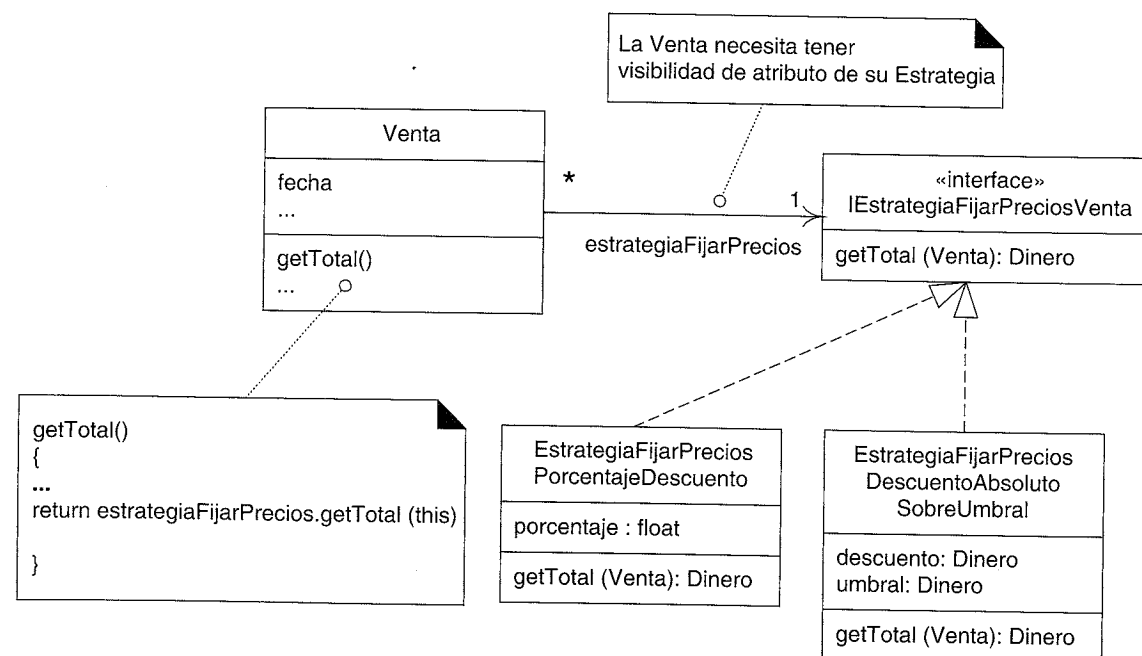


Figura 23.10. Los objetos de contexto necesitan tener visibilidad de atributo de su estrategia.

Creación de una Estrategia con una Factoría

Existen diferentes algoritmos o estrategias de fijación de precios, y cambian con el tiempo. ¿Quién debería crear la estrategia? Un enfoque directo es aplicar de nuevo el patrón Factoría: una *FactoriaDeEstrategiasFijarPrecios* puede ser la responsable de crear todas las estrategias (todos los algoritmos o las políticas conectables o cambiantes) que se necesitan en la aplicación. Como con la *FactoriaDeServicios*, se puede leer el nombre de la clase de implementación de la estrategia de fijación de precios como una propiedad del sistema (o alguna fuente de datos externa), y después crear una instancia. Con este *diseño dirigido por los datos* parcial (o *diseño reflexivo*) uno puede cambiar dinámicamente en cualquier momento —mientras se está ejecutando la aplicación del PDV NuevaEra— la política de fijación de precios, especificando la creación de una clase de Estrategia diferente.

Obsérvese que se utiliza una nueva factoría para las estrategias; es decir, diferente a la *FactoriaDeServicios*. Esto se ajusta al objetivo de Alta Cohesión —cada factoría está centrada de manera cohesiva en la creación de una familia de objetos relacionados—.

Notación UML: Observe que en la Figura 23.10 se referencia mediante una asociación directa a la interfaz *IEstrategiaFijarPreciosVenta*, no a una clase concreta. Eso indica que el atributo de referencia en la *Venta* se declarará en términos de la interfaz, no de una clase, de manera que el atributo se puede ligar con cualquier implementación de la interfaz.

Nótese que debido a que la política de fijación de precios cambia con frecuencia (podría ser cada hora), *no* es conveniente almacenar la instancia de la estrategia creada en un campo de la *FactoriaDeEstrategiasFijarPrecios*, sino crear una cada vez, leyendo la propiedad externa para obtener el nombre de la clase e instanciando luego la estrategia.

Y como con la mayoría de las factorías, la *FactoriaDeEstrategiasFijarPrecios* será un singleton (una instancia) y se accederá mediante el patrón Singleton (ver Figura 23.11).

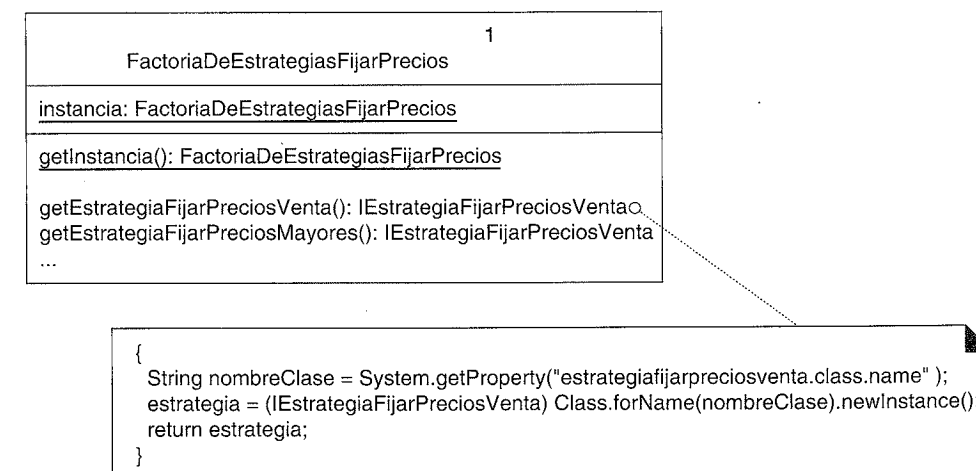


Figura 23.11. Factoría de estrategias.

Cuando se crea una instancia de la *Venta*, puede pedir a la factoría su estrategia de fijación de precios, como se muestra en la Figura 23.12.

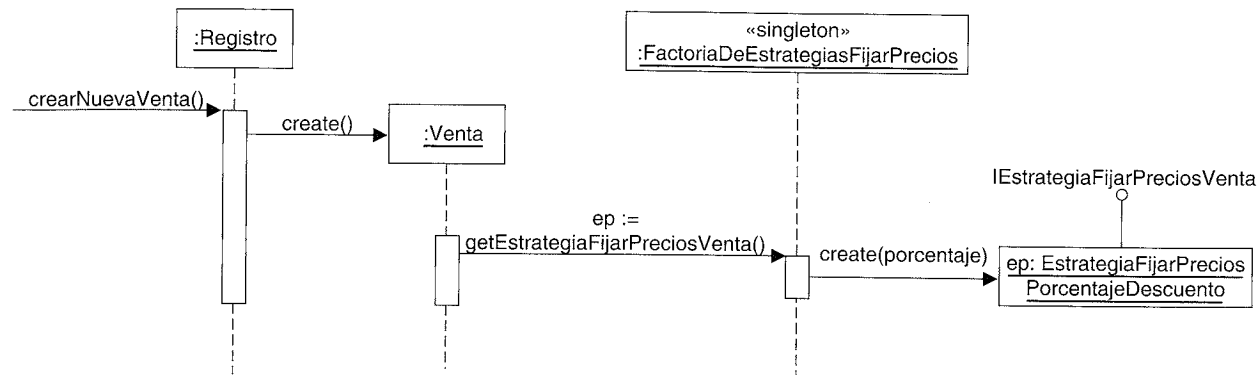


Figura 23.12. Creación de una estrategia.

Lectura e inicialización del valor del porcentaje

Finalmente, un problema de diseño que se ha ignorado hasta ahora es cómo encontrar los diferentes valores de los porcentajes o descuentos absolutos. Por ejemplo, el valor del porcentaje de la *EstrategiaFijarPreciosPorcentajeDescuento* podría ser 10% el lunes, pero 20% el martes.

Nótese también que el porcentaje de descuento podría estar relacionado con el tipo del comprador, como una persona mayor, en lugar de con un periodo de tiempo.

Estos números se almacenarán en algún almacén de datos externo, como una base de datos relacional, por lo que se pueden cambiar fácilmente. Luego, ¿qué objeto los leerá y asegurará que se asignan a la estrategia? Una opción razonable es la propia *FactoriaDeEstrategias*, puesto que crea la estrategia para fijar precios, y puede saber cuál es el porcentaje que hay que leer del almacén de datos (“el descuento actual del almacén”, “el descuento de las personas mayores”, etcétera).

Los diseños para la lectura de estos números procedentes de los almacenamientos de datos externos, varían desde los más simples a los más complejos, como una simple llamada SQL JDBC (por ejemplo, si se utilizan las tecnologías Java) o la colaboración con objetos que añaden niveles de indirección para ocultar la localización concreta, el lenguaje de consulta de los datos, o el tipo de almacén de datos. El análisis de los puntos de variación y evolución con respecto al almacén de datos revelará si es necesario que se proteja contra las variaciones. Por ejemplo, podríamos preguntar, “¿Estamos todos cómodos con el compromiso a largo plazo de utilizar una base de datos relacional que entienda SQL?”. Si es así, podría ser suficiente una simple llamada JDBC desde la *FactoriaDeEstrategias*.

Resumen

Con los patrones Estrategia y Factoría se ha conseguido Variaciones Protegidas con respecto a las políticas para fijar precios que varían dinámicamente. La Estrategia se fundamenta en el Polimorfismo y las interfaces para permitir algoritmos conectables en un diseño de objetos.

Patrones Relacionados

El patrón Estrategia se basa en el Polimorfismo, y proporciona Variaciones Protegidas con respecto a los algoritmos que cambian. Las Estrategias normalmente se crean mediante una Factoría.

23.7. Composite⁵ (GoF) y otros principios de diseño

Todavía surge otro interesante problema en los requisitos y el diseño: ¿cómo gestionamos el caso de varias políticas contradictorias de fijación de precios? Por ejemplo, suponga que una tienda tiene hoy (lunes) en vigor las siguientes políticas:

- Política de descuento del 20% a las personas mayores.
- Descuento del 15% en compras superiores a los 400 € para clientes preferentes.
- Los lunes, hay un 50% de descuento en las compras superiores a 500 €.
- Comprando una caja de té Darjeeling, obtiene el 15% de descuento en todo.

Suponga que una persona mayor que también es cliente preferente compra una caja de té Darjeeling, y 600 € de hamburguesas vegetales (claramente un vegetariano entusiasta al que le encanta el chai). ¿Qué política de fijación de precios se debería aplicar?

Explicándolo más claramente: ahora las estrategias de fijación de precios que se conectan a la venta dependen de tres factores:

- Periodo de tiempo (lunes).
- Tipo de cliente (persona mayor).
- Un producto concreto (té Darjeeling).

Otro punto que hay que aclarar: tres de las cuatro políticas del ejemplo son en realidad simplemente estrategias de “porcentaje de descuento”, lo que simplifica nuestra perspectiva del problema.

Parte de la respuesta a este problema requiere que se defina la **estrategia de resolución de conflictos** de la tienda. Normalmente, una tienda aplica “lo mejor para el cliente” (el precio más bajo) como estrategia de resolución de conflictos, pero no es obligatorio y podría cambiar. Por ejemplo, durante un periodo con dificultades financieras, la tienda podría verse obligada a utilizar la estrategia de resolución de conflictos “el precio más alto”.

El primer punto a tener en cuenta es que pueden existir múltiples estrategias coexistiendo al mismo tiempo, es decir, una venta podría tener asociadas varias estrategias para fijar el precio. Otro punto a destacar es que una estrategia de fijación de precios puede estar relacionada con el tipo de cliente (por ejemplo, una persona mayor). Esto afecta al diseño de la creación: la *FactoriaDeEstrategias* debe conocer el tipo del cliente en el momento de la creación de una estrategia de fijación de precios para el cliente.

Análogamente, una estrategia de fijación de precios puede estar relacionada con el tipo de producto que se compra (por ejemplo, té Darjeeling). Esto, del mismo modo, tiene implicaciones en el diseño de la creación: la *FactoriaDeEstrategias* debe conocer la *EspecificacionDelProducto* en el momento de la creación de una estrategia de fijación de precios influenciada por el producto.

⁵ N. del T.: No se ha traducido por el mismo motivo que en el caso del patrón Singleton.

¿Hay alguna forma de cambiar el diseño de manera que el objeto *Venta* no conozca si está tratando con una o más estrategias, y ofrecer también un diseño para la resolución de conflictos? Sí, con el patrón Composite.

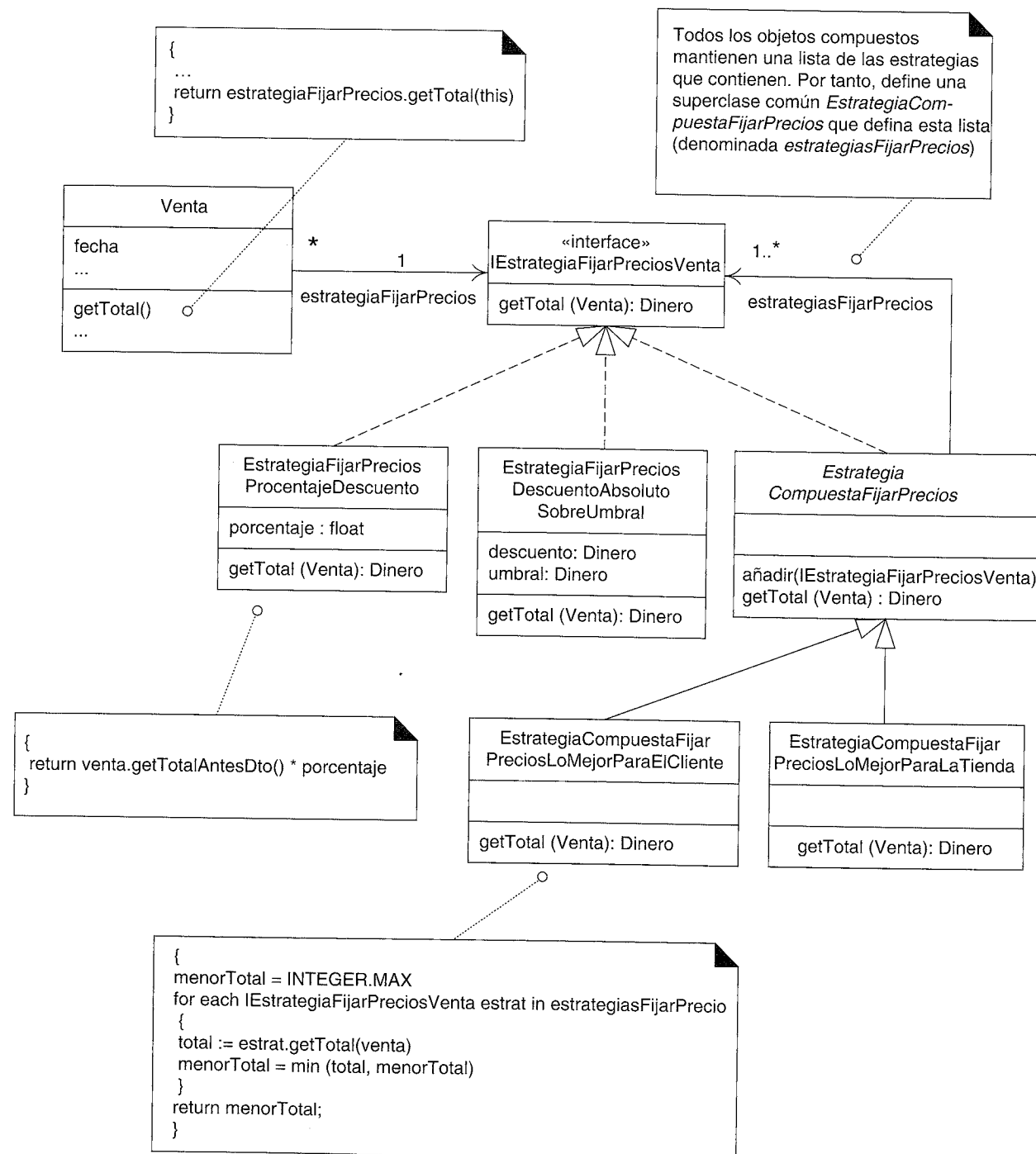


Figura 23.13. El patrón Composite.

Composite

Contexto/Problema

¿Cómo tratar un grupo o una estructura compuesta del mismo modo (polimórficamente) que un objeto no compuesto (atómico)?

Solución

Defina las clases para los objetos compuestos y atómicos de manera que implementen el mismo interfaz.

Por ejemplo, una nueva clase denominada *EstrategiaCompuestaFijarPreciosLoMejorParaElCliente* (bueno, por lo menos es descriptivo) puede implementar la *IEstrategiaFijarPreciosVenta* y ella misma contiene otros objetos *IEstrategiaFijarPreciosVenta*. La Figura 23.13 explica en detalle la idea de diseño.

Observe que en este diseño, la clase compuesta como *EstrategiaCompuestaFijarPreciosLoMejorParaElCliente* hereda el atributo *estrategiasFijarPrecios* que contiene una lista de más objetos *IEstrategiaFijarPreciosVenta*. Ésta es una característica distintiva de un objeto compuesto: el objeto compuesto externo contiene una lista de los objetos internos, y tanto los objetos externos como los internos implementan la misma interfaz. Es decir, la propia clase compuesta implementa la interfaz *IEstrategiaFijarPreciosVenta*.

Por tanto, podemos conectar al objeto *Venta* tanto a un objeto *EstrategiaCompuestaFijarPreciosLoMejorParaElCliente* (que contiene otras estrategias dentro de él) o a un objeto atómico *EstrategiaFijarPreciosPorcentajeDescuento*, y la *Venta* no conoce o no se preocupa si su estrategia de fijación de precios es atómica o compuesta —parecen iguales para el objeto *Venta*—. Es simplemente otro objeto que implementa la interfaz *IEstrategiaFijarPreciosVenta* y entiende el mensaje *getTotal* (Figura 23.14).

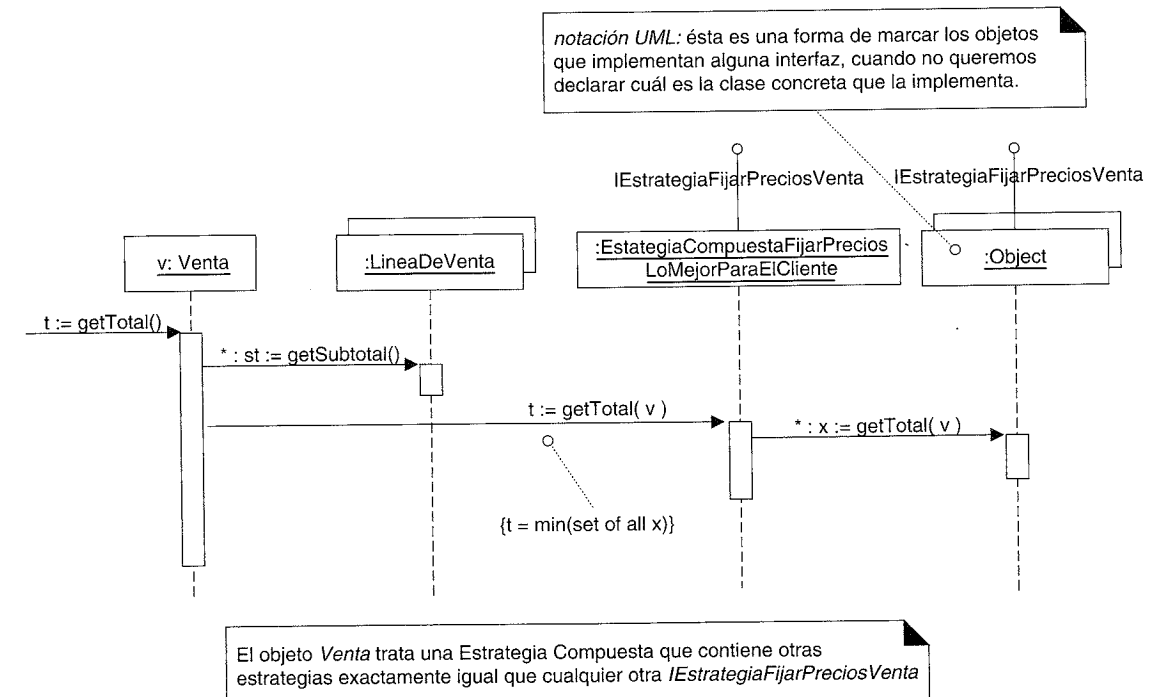


Figura 23.14. Colaboración con un Composite.

Notación UML: En la Figura 23.14, nótese, por favor, el modo de indicar los objetos que implementan una interfaz, cuando no nos preocupa especificar la clase de implementación exacta. Simplemente especificando la clase de implementación como *Object* queremos decir “sin comentarios” acerca de la clase específica. Ésta es una necesidad habitual en la elaboración de diagramas.

A continuación definimos la *EstrategiaCompuestaFijarPrecios* y una de sus subclases para aclarar la explicación con una muestra de código en Java:

```
//superclase luego todas las subclases heredan una List de estrategias

public abstract class EstrategiaCompuestaFijarPrecios
    implements IEstrategiaFijarPreciosVenta
{
    protected List estrategiasFijarPrecios = new ArrayList();

    public añadir(IEstrategiaFijarPreciosVenta e)
    {
        estrategiasFijarPrecios.add(e);
    }

    public abstract Dinero getTotal( Venta v );
}

//final de la clase

//Una Estrategia Compuesta que devuelve el total más pequeño
//de sus instancias EstrategiaFijarPreciosVenta internas

public class EstrategiaCompuestaFijarPreciosLoMejorParaElCliente
    extends EstrategiaCompuestaFijarPrecios
{
    public Dinero getTotal(Venta venta)
    {
        Dinero menorTotal = new Dinero (Integer.MAX_VALUE);

        //iteramos sobre todas las estrategias internas

        for( Iterator i = estrategiasFijarPrecios.iterator(); i.hasNext(); )
        {
            IEstrategiaFijarPreciosVenta estrategia =
                (IEstrategiaFijarPreciosVenta)i.next();
            Dinero total = estrategia.getTotal(venta);
            menorTotal = total.min(menorTotal);
        }
        return menorTotal();
    }
}

//final de la clase
```

Notación UML: la Figura 23.13 introduce nueva notación UML para representar las jerarquías de clases y la herencia, que se explica en la Figura 23.15.

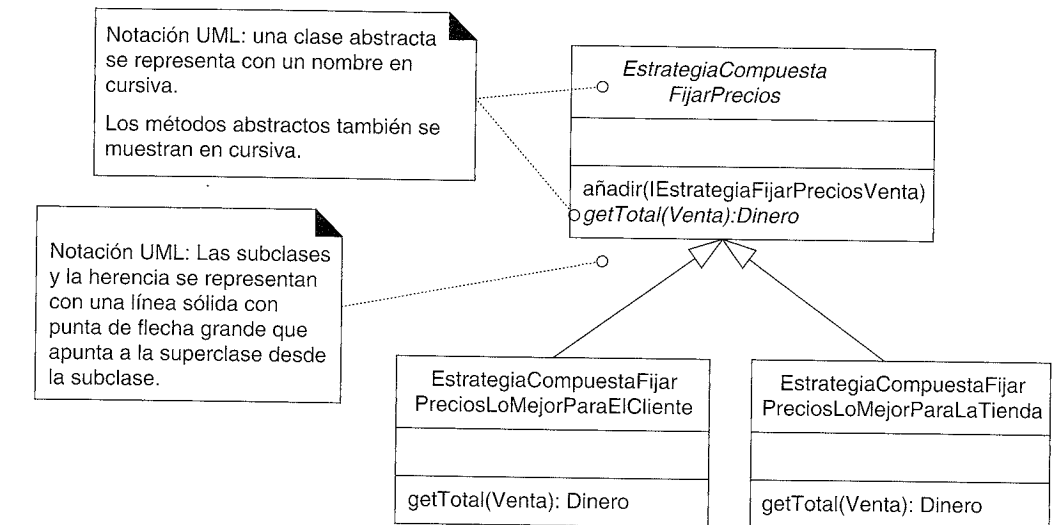


Figura 23.15. Superclases abstractas, métodos abstractos y herencia en UML.

Creación de múltiples instancias EstrategiaFijarPreciosVenta

Con el patrón Composite, hemos creado un grupo de diferentes (y contradictorias) estrategias de fijación de precios, que para *Venta* aparecen como una única estrategia. El objeto compuesto que contiene el grupo también implementa la interfaz *IEstrategiaFijarPreciosVenta*. La parte más desafiante (e interesante) de este problema de diseño es: ¿cuándo creamos estas estrategias?

Un diseño deseable comenzará creando un Composite que contenga las políticas de descuento de la tienda en el momento actual (que podría ser 0% de descuento si no hay ninguna activa), como alguna *EstrategiaFijarPreciosPorcentajeDescuento*. Entonces, si en un paso posterior del escenario, se descubre que se debe aplicar otra estrategia para fijar precios (como el descuento a las personas mayores), se añadirá fácilmente al objeto compuesto utilizando el método heredado *EstrategiaCompuestaFijarPrecios.añadir*.

Existen tres puntos en el escenario donde se podrían agregar al objeto compuesto las estrategias de fijación de precios:

1. Descuento actual a nivel de tienda, se añade cuando se crea la venta.
2. Descuento por el tipo de cliente, se añade cuando se informa al PDV del tipo de cliente.
3. Descuento según el tipo de producto (si compra el té Darjeeling obtendrá un descuento del 15% sobre el total de la venta), se añade cuando se introduce el producto en la venta.

El diseño del primer caso se muestra en la Figura 23.16. Como en el diseño original que se discutió anteriormente, el nombre de la clase estrategia que se va a instanciar se podría leer como una propiedad del sistema, y el valor del porcentaje se podría leer desde un almacenamiento de datos externo.

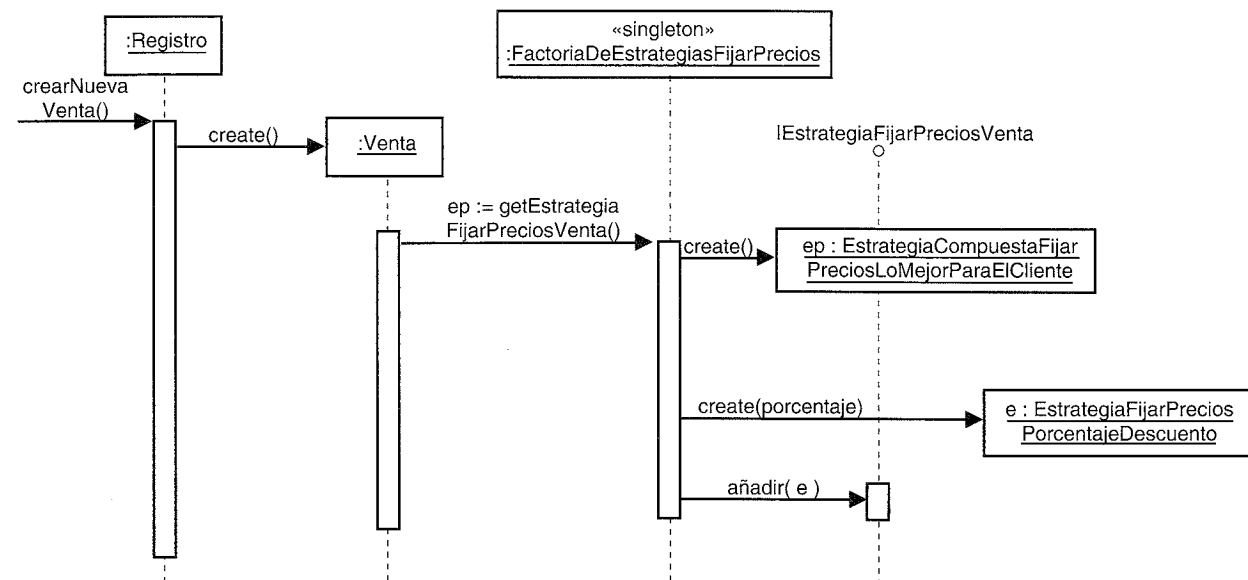


Figura 23.16. Creación de una estrategia compuesta.

Para el segundo caso de descuento según el tipo de cliente, en primer lugar recordemos la extensión del caso de uso que identificó anteriormente este requisito:

Caso de uso UC1: Procesar Venta

... Extensiones (o Flujos Alternativos):

- 5b. El Cliente dice que le son aplicables descuentos (ej. empleado, cliente preferente):
1. El Cajero señala la petición de descuento.
 2. El Cajero introduce la identificación del Cliente.
 3. El Sistema presenta el descuento total, basado en las reglas de descuento.

Esto indica una nueva operación del sistema en el sistema de PDV, además de *crearNuevaVenta*, *introducirArticulo*, *finalizarVenta* y *realizarPago*. Llamaremos a esta quinta operación del sistema *introducirClienteParaDescuento*; opcionalmente podría tener lugar después de la operación *finalizarVenta*. Esto implica que se deberá introducir algún tipo de identificación del cliente a través de la interfaz de usuario, el *clienteID*. Quizás podría capturarse mediante un lector de tarjetas o a través del teclado.

El diseño de este segundo caso se muestra en las Figuras 23.17 y 23.18. No sorprende que el objeto factoría sea el responsable de la creación de la estrategia de fijación de precios adicional. Podría crear otra *EstrategiaFijarPreciosPorcentajeDescuento* que represente, por ejemplo, un descuento para las personas mayores. Pero como con el di-

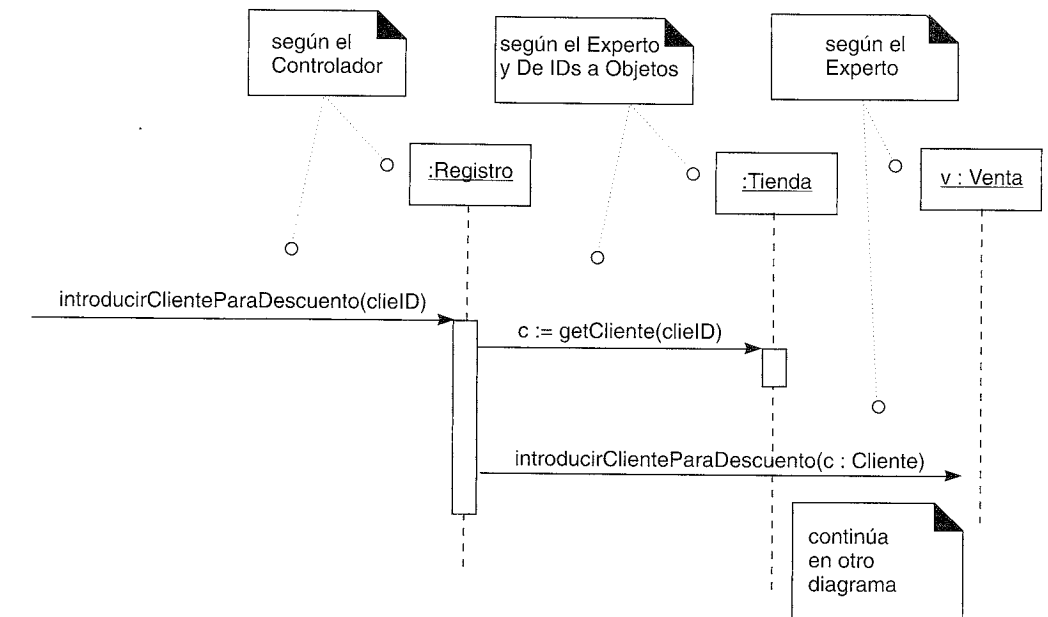


Figura 23.17. Creación de la estrategia de fijación de precios para el descuento de los clientes, parte 1.

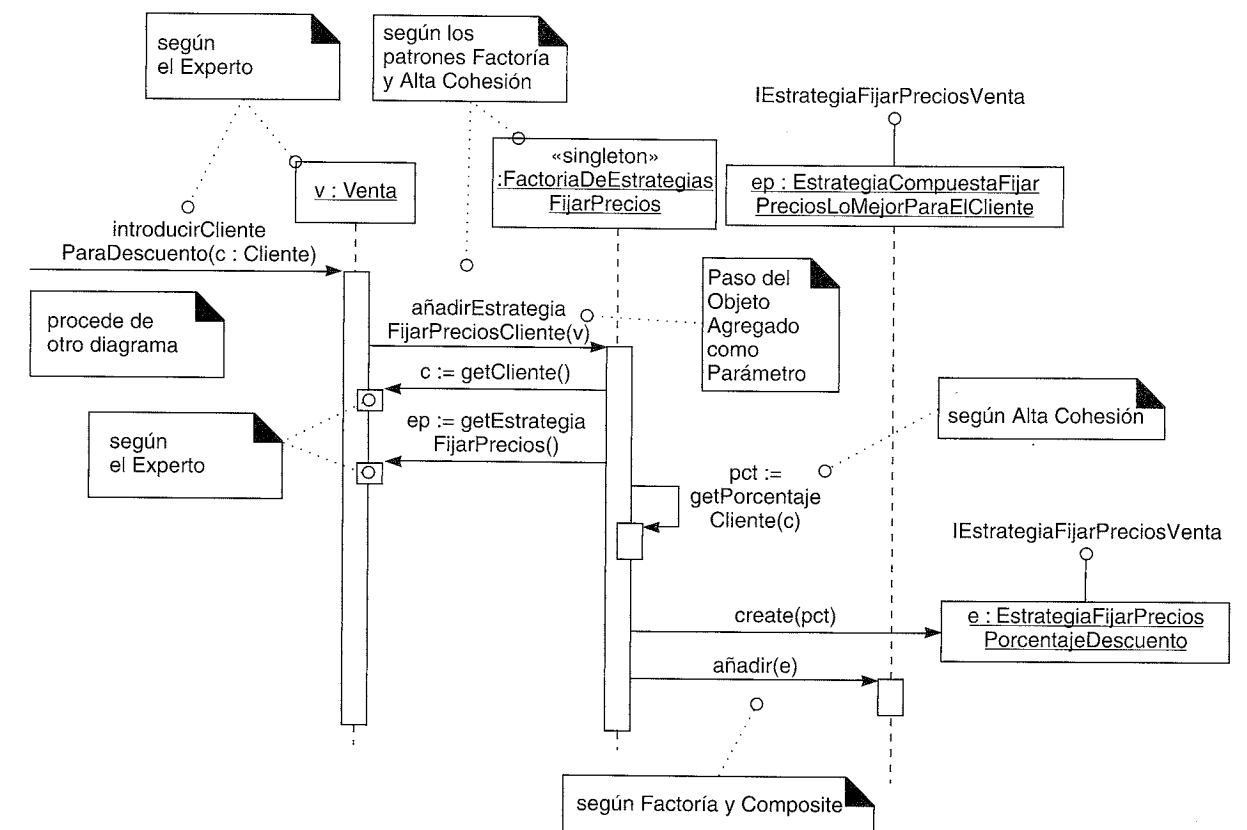


Figura 23.18. Creación de la estrategia de fijación de precios para el descuento de los clientes, parte 2.

seño de la creación original, la elección de la clase se leerá como una propiedad del sistema, al igual que el porcentaje específico para el tipo de cliente, para proporcionar Variaciones Protegidas con respecto a los cambios de las clases o los valores. Nótese que en virtud del patrón Composite, la *Venta* podría tener conectadas dos o tres estrategias contradictorias de fijación de precios, pero sigue pareciendo una única estrategia para el objeto *Venta*.

Notación UML: Las Figuras 23.17 y 23.18 muestran una importante idea UML sobre los diagramas de interacción: la división de un diagrama en dos, para que cada uno de ellos sea más legible.

Consideración de los patrones GRASP y otros principios en el diseño

Revisemos el razonamiento en función de algunos de los patrones GRASP básicos: para este segundo caso, ¿por qué no envía el *Registro* un mensaje a la *FactoriaDeEstrategiasFijarPrecios*, para crear esta nueva estrategia de fijación de precios y pasársela entonces a la *Venta*? Uno de los motivos es para mantener Bajo Acoplamiento. La *Venta* ya está acoplada a la factoría; si hacemos que el *Registro* también colabore con ella, incrementaríamos el acoplamiento en el diseño. Además, la *Venta* es el Experto en Información que conoce su estrategia actual de fijación de precios (que se va a modificar); luego, según el Experto, está justificado que se delegue en la *Venta*.

Obsérvese en el diseño que el *clienteID* se transforma en un objeto *Cliente*, preguntando el *Registro* a la *Tienda* por un *Cliente*, dado un ID. En primer lugar, se puede justificar que se otorgue a la *Tienda* la responsabilidad *getClientes*; de acuerdo con el Experto en Información y el objetivo de salto en la representación reducido, la *Tienda* puede conocer a todos los *Clientes*. Y el *Registro* pregunta a la *Tienda*, porque el *Registro* ya tiene visibilidad de atributo de la *Tienda* (a partir del trabajo de diseño previo); si la *Venta* tuviera que preguntar a la *Tienda*, la *Venta* necesitaría una referencia a la *Tienda*, lo que incrementaría el acoplamiento más allá de los niveles actuales y, por tanto, no mantendría Bajo Acoplamiento.

Transformación de IDs en objetos

Segundo, ¿por qué transformar *clienteID* (un “ID” —quizás un número—) en un objeto *Cliente*? Ésta es una práctica habitual del diseño de objetos —transformar claves e identificadores (IDs) de las cosas en verdaderos objetos—. A menudo esta transformación tiene lugar poco después de que se introduzca el ID o la clave en la capa del dominio del Modelo de Diseño desde la capa de UI. No tiene nombre de patrón, pero podría ser un candidato puesto que es un estilo habitual entre los diseñadores de objetos con experiencia —quizás *De IDs a Objetos*—. ¿Por qué preocuparse? Porque tener un auténtico objeto *Cliente* que encapsula un conjunto de información sobre el cliente, y que puede tener comportamiento (relacionado con el Experto en Información, por ejemplo), con frecuencia se vuelve beneficioso y flexible cuando crece el diseño, incluso si el diseñador no creyó que fuera necesario un verdadero objeto y pensó en cambio que un simple número sería suficiente. Nótese que en el diseño inicial, la transformación del *articuloID* en un objeto *EspecificacionDelProducto* es otro ejemplo del patrón *De IDs a Objetos*.

Paso de objetos agregados como parámetros

Finalmente, observe que en el mensaje *añadirEstrategiaFijarPrecioCliente(v:Venta)* pasamos una *Venta* a la factoría, y entonces la factoría se vuelve y pregunta a la *Venta* por el *Cliente* y su *EstrategiaFijarPrecios*.

¿Por qué no extraer exactamente estos dos objetos de la *Venta*, y, en lugar de lo anterior, pasar a la factoría el *Cliente* y la *EstrategiaFijarPrecios*? La respuesta es otro estilo del diseño de objetos típico: evitar extraer los objetos hijos de un objeto padre o agregado, y entonces pasar los objetos hijos. Mejor, pase el objeto agregado que contiene los hijos.

Si se sigue este principio se incrementa la flexibilidad, porque entonces la factoría puede colaborar con la *Venta* completa de algunas maneras que no habíamos anticipado que fueran necesarias previamente (que es muy normal), y como corolario, reduce la necesidad de anticipar lo que necesita el objeto factoría; el diseñador pasa exactamente la *Venta* completa, sin conocer los objetos más específicos que podría necesitar la factoría. Aunque este estilo no tiene nombre, está relacionado con el Bajo Acoplamiento y Variaciones Protegidas. Quizás podría llamarse patrón *Paso de Objetos Agregados como Parámetro*.

Resumen

De este problema de diseño se han extraído muchos consejos de diseño de objetos. Un diseñador de objetos experto ha incorporado a su memoria muchos de estos patrones estudiando las explicaciones que se han publicado, y ha asimilado los principios fundamentales, como los que se describen en la familia GRASP.

Por favor, obsérvese que aunque esta explicación del Composite se ha realizado para una familia de Estrategias, el patrón Composite se puede aplicar a otros tipos de objetos, no sólo estrategias. Por ejemplo, es habitual que se creen “macro commands” —commands que contienen otros commands— mediante el uso del Composite. El patrón Command se describirá en un capítulo posterior.

Ejercicios Propuestos

Ejercicio 1

La compra de un producto específico da lugar a un nuevo descuento en la venta completa. Por ejemplo, si compró té Darjeeling, el 15% de descuento en el total de la venta.

Ejercicio 2

Todas las políticas de fijación de precios que se han considerado hasta el momento se aplican sobre el total de la venta, se denominan algunas veces descuentos a nivel de transacción. Pero el reto de diseño más interesante es gestionar los descuentos de las líneas de venta. Por ejemplo:

- Comprando dos trajes, obtenga uno gratis.
- Comprando tres ordenadores X, obtenga un descuento del 50% en la impresora Y.

¿Hay una forma elegante de diseñar esto con los objetos Estrategia?

Patrones Relacionados

El patrón Composite se utiliza normalmente junto con los patrones Estrategia y Command. El Composite se basa en el Polimorfismo y proporciona Variaciones Protegidas a los clientes de manera que no les afecta si el objeto con el que se relacionan es atómico o compuesto.

23.8. Fachada (GoF)

Otro requisito que se ha elegido para esta iteración es dar soporte a *reglas de negocio conectables*. Es decir, en puntos predecibles de los escenarios, como cuando tienen lugar *crearNuevaVenta* e *introducirArticulo* en el caso de uso *Procesar Venta*, o cuando un cajero comienza a vender, a distintos clientes que deseen comprar el PDV NuevaEra les gustaría adaptar ligeramente su comportamiento.

Siendo más precisos, asuma que se desea que las reglas puedan invalidar una acción. Por ejemplo:

- Suponga que cuando se crea una nueva venta, es posible identificar que se pagará mediante un vale-regalo (lo que es posible y habitual). Entonces, una tienda podría tener una regla para permitir que sólo se compre un único artículo si se utiliza un vale-regalo. En consecuencia, deberían invalidarse las operaciones *introducirArticulo* que sigan a la primera.
- Si se paga la venta mediante un vale-regalo, se invalidan todos los tipos de devoluciones de dinero al cliente excepto otro vale-regalo. Por ejemplo, si el cajero solicita el cambio en dinero en efectivo o como crédito para la cuenta del cliente en la tienda, se deben invalidar estas peticiones.
- Suponga que cuando se crea una nueva venta es posible identificar que es una donación benéfica (de la tienda a una ONG). La tienda podría tener una regla que sólo permitiera la entrada de artículos de valor menor a 250 € cada uno, y también añadir sólo artículos a la venta si el “cajero” que inició la sesión es un encargado.

En cuanto al análisis de requisitos, se deben identificar los puntos concretos del escenario en todos los casos de uso (*introducirArticulo*, *elegirCambioEnEfectivo*,...). Para este estudio, sólo se considerará el punto *introducirArticulo*, pero se puede aplicar la misma solución a todos los puntos.

Suponga que el arquitecto software quiere un diseño que afecte poco a los componentes software que ya existen. Es decir, quiere diseñar separando los intereses, y factorizar esta regla en un interés separado. Es más, suponga que el arquitecto no está seguro de cuál es la mejor implementación para gestionar esta regla conectable, y podría querer experimentar con diferentes soluciones para representar, cargar y evaluar las reglas. Por ejemplo, las reglas se pueden implementar siguiendo el patrón Estrategia, o con intérpretes de reglas de libre distribución que leen e interpretan un conjunto de reglas IF-THEN, o con intérpretes de reglas comerciales, que hay que comprar, entre otras soluciones.

Para solucionar este problema de diseño, se puede utilizar el patrón Fachada.

Fachada (Facade)

Contexto/Problema

Se requiere una interfaz común, unificada para un conjunto de implementaciones o interfaces dispares —como en un subsistema—. Podría no ser conveniente acoplarla con muchas cosas del subsistema, o la implementación del subsistema podría cambiar. ¿Qué hacemos?

Solución

Defina un único punto de conexión con el subsistema —un objeto fachada que envuelve al subsistema—. Este objeto fachada presenta una única interfaz unificada y es responsable de colaborar con los componentes del subsistema.

Una Fachada es un objeto “front-end” que es el único punto de entrada para los servicios de un subsistema⁶; la implementación y otros componentes del subsistema son privados y no pueden verlos los componentes externos. La Fachada proporciona Variaciones Protegidas frente a los cambios en las implementaciones de un subsistema.

Por ejemplo, definiremos un subsistema “motor de reglas”, cuya implementación específica no se conoce todavía. Será responsable de evaluar un conjunto de reglas contra una operación (mediante alguna implementación oculta), e indicar entonces si alguna de las reglas invalida la operación.

El objeto fachada de este subsistema se llamará *FachadaMotorReglasPDV*. El diseñador decide colocar las llamadas a esta fachada cerca del comienzo de los métodos que se han definido como los puntos para las reglas conectables, como en este ejemplo:

```
public class Venta
{

    public void crearLineaDeVenta( EspecificacionDelProducto espec, int
        cantidad)
    {
        LineaDeVenta ldv = new LineaDeVenta(espec, cantidad);

        //llamada a la fachada
        if (FachadaMotorReglasPDV.getInstancia().esInvalido (ldv,this))
            return;

        lineasDeVenta.add(ldv);
    }

    //...

} //final de la clase
```

Véase el uso del patrón Singleton. Normalmente se accede a la Fachada por medio del Singleton.

Con este diseño, la complejidad y la implementación del modo en el que se representarán y evaluarán las reglas se oculta en el subsistema del “motor de reglas”, al que se accede por medio de la fachada *FachadaMotorReglasPDV*. Obsérvese que el subsistema que oculta el objeto fachada podría contener docenas o cientos de clases de objetos, o incluso una solución no orientada a objetos, únicamente como cliente del subsistema sólo vemos su único punto de acceso público.

Y se ha conseguido una separación de intereses en cierta medida — se han delegado en otro subsistema todas las cuestiones de manejo de reglas—.

Resumen

El patrón Fachada es sencillo y se utiliza ampliamente. Oculta un subsistema detrás de un objeto.

⁶ El término “subsistema” se está utilizando en un sentido informal para designar a un grupo de componentes relacionados, no exactamente como se define en UML.

Ejercicios Propuestos

Ejercicio 1

Diseñe la gestión de las reglas con el patrón Estrategia, cuyos nombres de clase se obtienen dinámicamente leyendo de una fuente externa.

Ejercicio 2

Si se implementa en Java, diseñe la gestión de las reglas con Jess, un intérprete de reglas de libre distribución para uso con fines académicos disponible en <http://herzberg.ca.sandia.gov/jess/>

Patrones Relacionados

Normalmente se accede a las fachadas por medio del patrón Singleton. Los dos proporcionan Variaciones Protegidas de la implementación de un subsistema, añadiendo un objeto de Indirección que ayuda a mantener Bajo Acoplamiento. Los objetos externos se acoplan a un punto del subsistema: el objeto fachada.

Como se describe en el patrón Adaptador, un objeto adaptador puede utilizarse para envolver el acceso a sistemas externos que tienen interfaces diferentes. Esto es una clase de fachada pero el énfasis está en proporcionar adaptación a interfaces diferentes, y por ello se llama más específicamente un adaptador.

Notación UML: UML proporciona una notación para agrupaciones de propósito general denominadas **paquetes**, cuyo icono es un tipo de carpeta etiquetada. Los paquetes se pueden utilizar para mostrar agrupaciones lógicas de objetos; se podría corresponder con algo como los paquetes de Java o los *namespaces* de C++, o con otros componentes agregados o subsistemas lógicamente distintos. Nótese que en la Figura 23.19 sólo la *FachadaMotorReglasPDV* es pública con respecto a su paquete.

Existe notación UML más compleja para representar subsistemas, pero la notación de la Figura 23.19 será suficiente por ahora. El diseño con paquetes se estudiará con más detalle en la siguiente iteración.

23.9. Observador/Publicar-Suscribir/Modelo de Delegación de Eventos (GoF)

Otro requisito de la iteración es añadir la capacidad de que una ventana GUI actualice la información que muestra sobre el total de la venta cuando éste cambia (ver Figura 23.20). La idea es solucionar el problema para este caso particular, y después en iteraciones posteriores, extender la solución para actualizar la información de la GUI también para los cambios de otros datos.

¿Por qué no hacer lo siguiente como solución? Cuando la *Venta* cambia su total, el objeto *Venta* envía un mensaje a la ventana, pidiéndole que actualice la información que muestra.

Recordemos que el principio de Separación Modelo-Vista disuade de tales soluciones. Establece que los objetos del “modelo” (objetos que no pertenecen al UI como la *Venta*) no deberían conocer los objetos de la vista o presentación tales como una ventana. Dicho principio fomenta Bajo Acoplamiento entre los objetos de otras capas y los de la capa de presentación (UI).

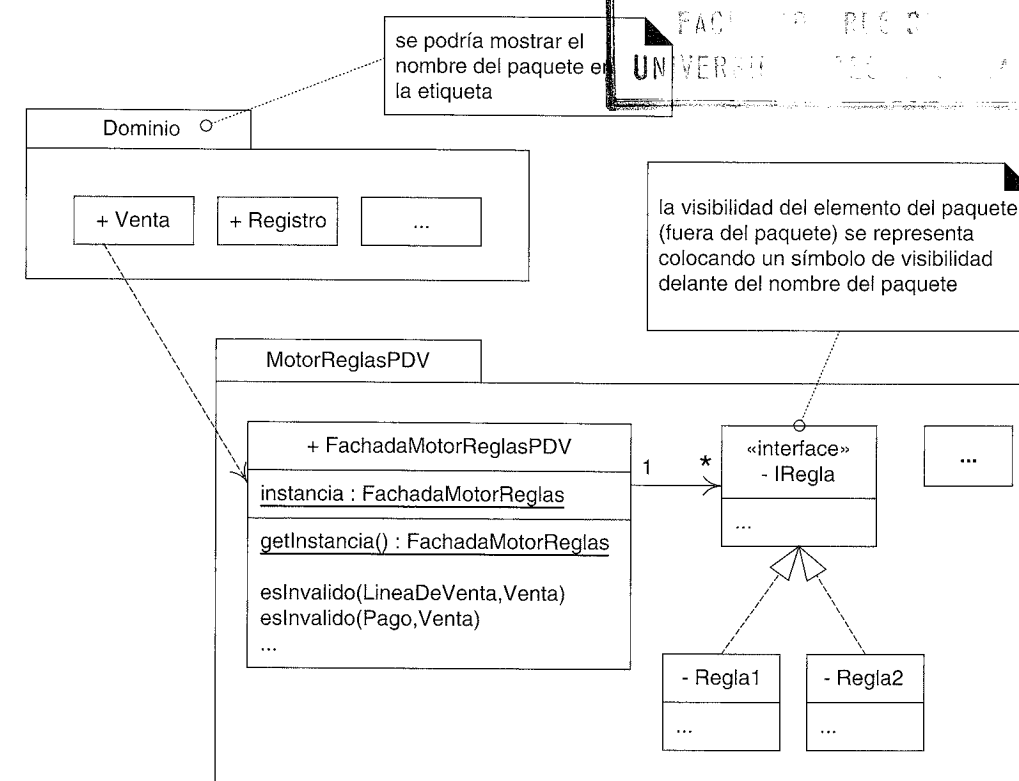


Figura 23.19. Notación de paquetes UML.

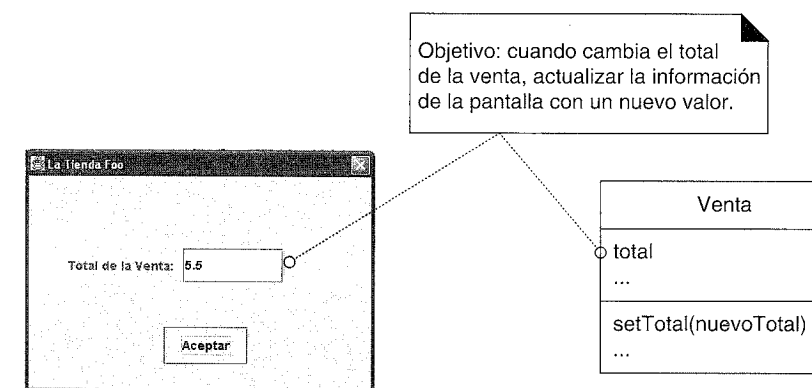
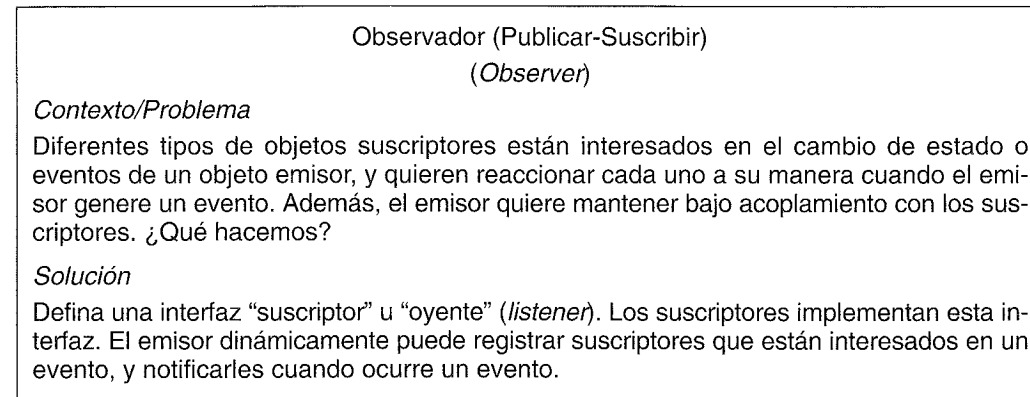


Figura 23.20. Actualización de la interfaz cuando cambia el total de la venta.

Una consecuencia de mantener este bajo acoplamiento es que permite reemplazar la vista o capa de presentación por una nueva, o una ventana concreta por otra nueva, sin afectar a los objetos que no pertenecen a la UI. Si los objetos del modelo no conocen los objetos Swing de Java (por ejemplo), entonces es posible desconectar una interfaz Swing, o desconectar una ventana concreta, y conectar algo más.

Por tanto, la Separación Modelo-Vista mantiene Variaciones Protegidas con respecto a los cambios en la interfaz de usuario.

Para solucionar este problema de diseño se puede utilizar el patrón Observador.



Una solución de ejemplo se describe en detalle en la Figura 23.21.

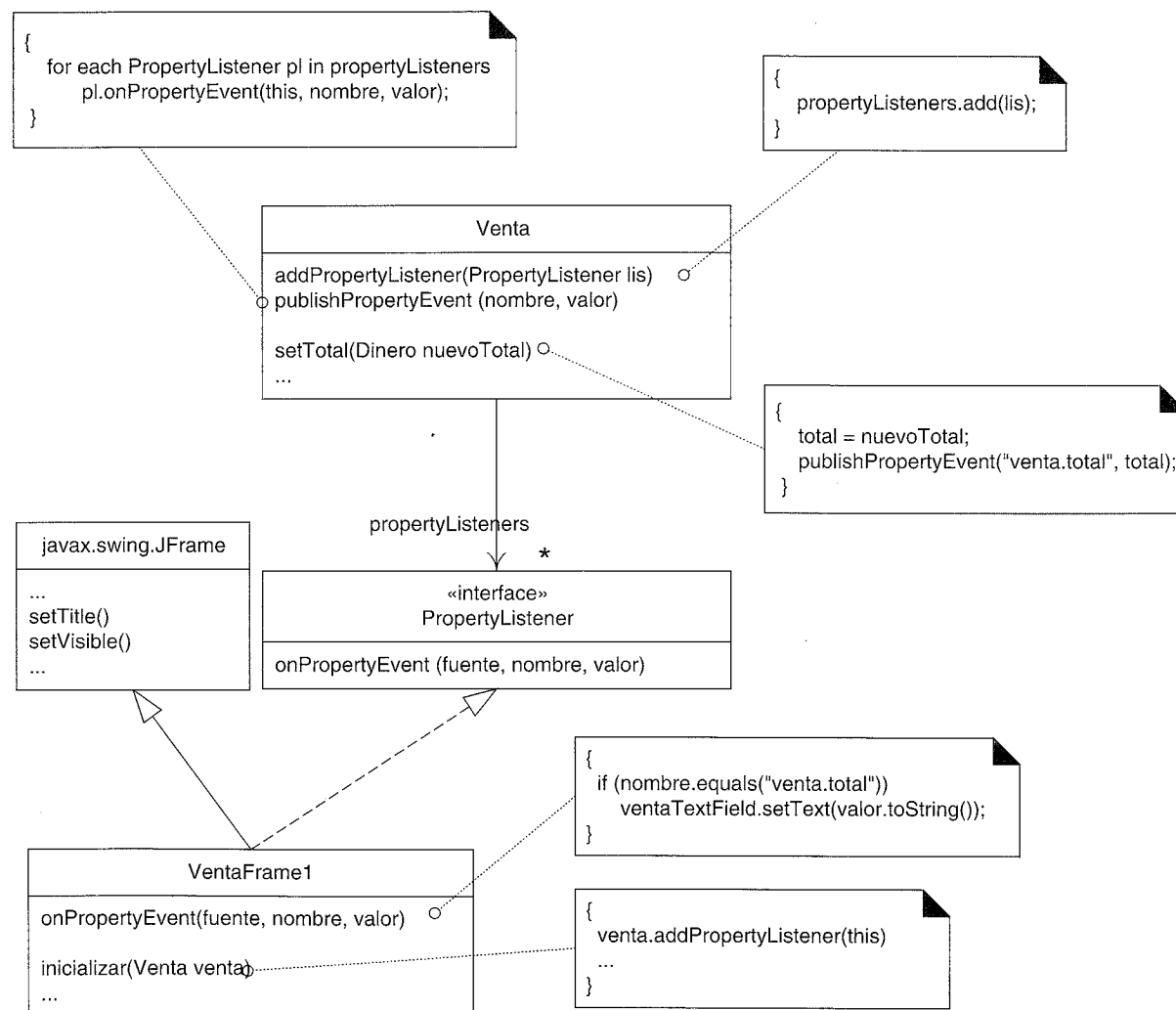


Figura 23.21. El patrón Observador.

Las ideas y pasos fundamentales de este ejemplo son:

1. Se define una interfaz; en este caso *PropertyListener*⁷ con la operación *onPropertyEvent*.
2. Se define la ventana que implementa la interfaz.
 - *VentaFrame1* implementará el método *onPropertyEvent*.
3. Cuando se inicializa la ventana *VentaFrame1*, se le pasa la instancia de *Venta* de la que está mostrando el total.
4. La ventana *VentaFrame1* se registra o suscribe a la instancia de *Venta* para que le notifique acerca de los “eventos sobre la propiedad”, por medio del mensaje *addPropertyListener*. Es decir, cuando una propiedad (como el total) cambia, la ventana quiere que se le notifique.
5. Observe que la *Venta* no conoce los objetos *VentaFrame1*; más bien sólo conoce los objetos que implementan la interfaz *PropertyListener*. Esto disminuye el acoplamiento entre la *Venta* y la ventana —se acopla sólo con una interfaz, no con la clase de la GUI—.
6. Por tanto, la instancia de *Venta* es un *emisor* de “eventos sobre la propiedad”. Cuando cambia el total, itera sobre todos los objetos *PropertyListener* que están suscritos, y se lo notifica a cada uno.

Notación UML: Obsérvese en la Figura 23.21 que a los métodos interesantes se les añaden notas de comentario que muestran la implementación. Estas notas añaden información sobre el comportamiento dinámico a un diagrama de tipos estáticos. En algunos casos, un diagrama de clases con estas notas puede sustituir la necesidad de diagramas de interacción adicionales. Esto no significa que estemos aconsejando que se eviten los diagramas de interacción, sino que indica enfoques de notación alternativos.

El objeto *VentaFrame1* es el observador/suscriptor/oyente. En la Figura 23.22, *suscribe* su interés en los eventos sobre las propiedades de la *Venta*, que es un *emisor* de

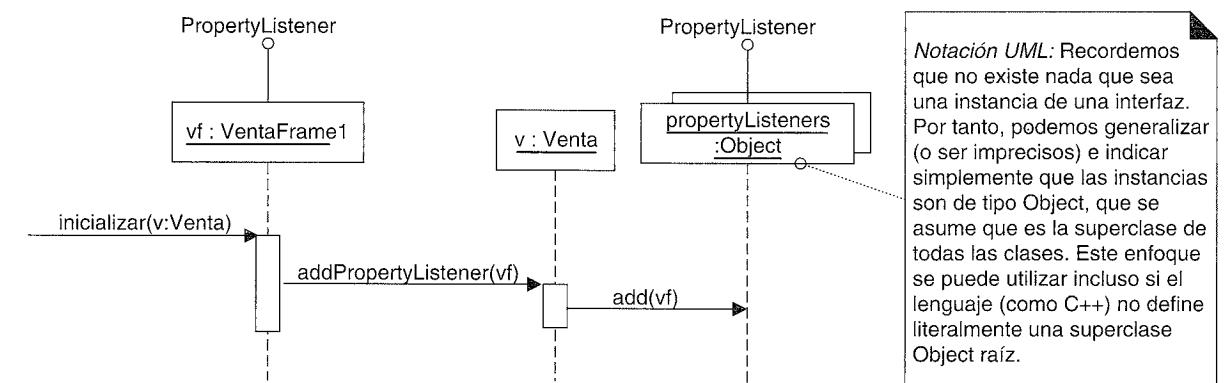


Figura 23.22. El observador *VentaFrame1* se suscribe al emisor *Venta*.

⁷ N. del T.: No se han traducido los nombres de los métodos en los ejemplos de esta sección ya que corresponden a la implementación en Java del patrón.

eventos sobre la propiedad. La *Venta* añade el objeto a su lista de suscriptores de tipo *PropertyListener*. Nótese que la *Venta* no conoce a *VentaFrame1* como un objeto *VentaFrame1* sino como un objeto *PropertyListener*; esto disminuye el acoplamiento entre la capa del modelo y la vista.

Como se ilustra en la Figura 23.23, cuando cambia el total de la *Venta*, itera sobre todos los suscriptores que se han registrado, y “publica un evento” enviando a cada uno el mensaje *onPropertyEvent*.

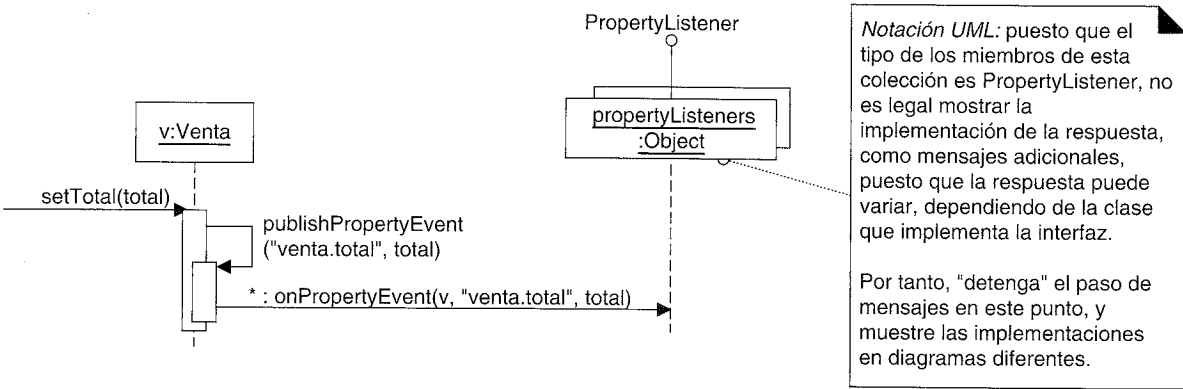


Figura 23.23. La *Venta* publica un evento sobre la propiedad a todos sus suscriptores.

Notación UML: Observe el enfoque para gestionar los mensajes polimórficos en un diagrama de interacción, en la Figura 23.23.

VentaFrame1, que implementa la interfaz *PropertyListener*, por tanto implementa el método *onPropertyEvent*. Cuando el *VentaFrame1* recibe el mensaje, envía un mensaje a su objeto *TextField* que es un elemento gráfico de la GUI para actualizar el nuevo total de la venta. Ver Figura 23.24.

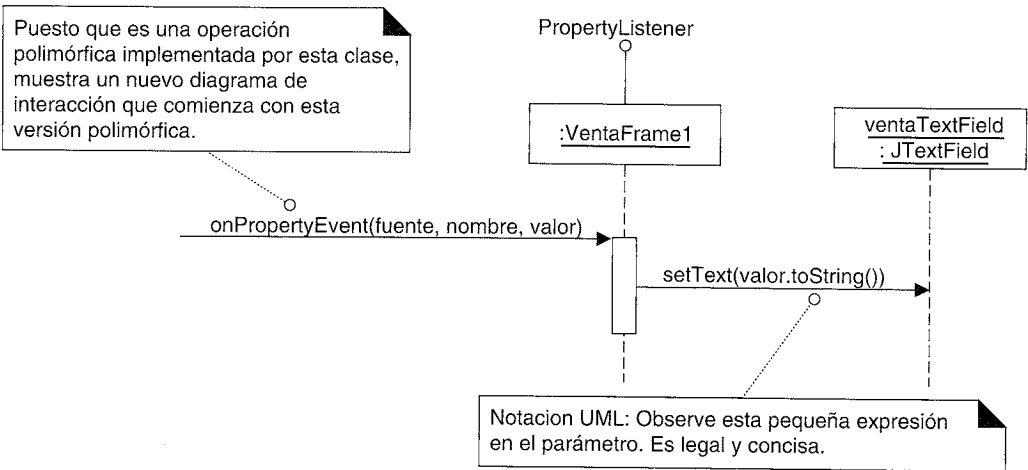


Figura 23.24. El suscriptor *VentaFrame1* recibe la notificación de un evento publicado.

En este patrón, existe todavía algo de acoplamiento entre el objeto del modelo (la *Venta*) y el objeto de la vista (el *VentaFrame1*). Pero se trata de un acoplamiento débil con

una interfaz independiente de la capa de presentación —la interfaz *PropertyListener*—. Y el diseño realmente no necesita que se registre ningún objeto suscriptor en el emisor (ningún objeto tiene que estar escuchando). Es decir, la lista de objetos *PropertyListener* que se registran en la *Venta* puede estar vacía. Resumiendo, el acoplamiento con una interfaz genérica de objetos que no necesita estar presente y a la que se le pueden añadir (y eliminar) elementos dinámicamente, mantiene bajo acoplamiento. Por tanto, se ha conseguido Variaciones Protegidas con respecto a los cambios en la interfaz de usuario, mediante el uso de una interfaz y el polimorfismo.

¿Por qué se le denomina Observador, Publicar-Suscribir, o Modelo de Delegación de Eventos?

Originalmente, este estilo se llamó publicar-suscribir, y todavía se le conoce ampliamente por este nombre. Un objeto “publica eventos”, como la *Venta* que publica el “evento sobre la propiedad” cuando cambia el total. Podría pasar que ningún objeto estuviera interesado en este evento, en cuyo caso, la *Venta* no tendría ningún suscriptor registrado. Pero los objetos que están interesados, “suscriben” o registran su interés en un evento pidiéndole al emisor que le notifique. Esto se hizo con el mensaje *Venta--addPropertyListener*. Cuando tiene lugar el evento, se notifica a los suscriptores que están registrados mediante un mensaje.

Se le ha llamado Observador porque el oyente o suscriptor está observando el evento; ese término se hizo popular en Smalltalk a principio de los ochenta.

También se le ha denominado Modelo de Delegación de Eventos (en Java) porque el emisor delega la gestión de los eventos a los “oyentes” (suscriptores; ver Figura 23.25).

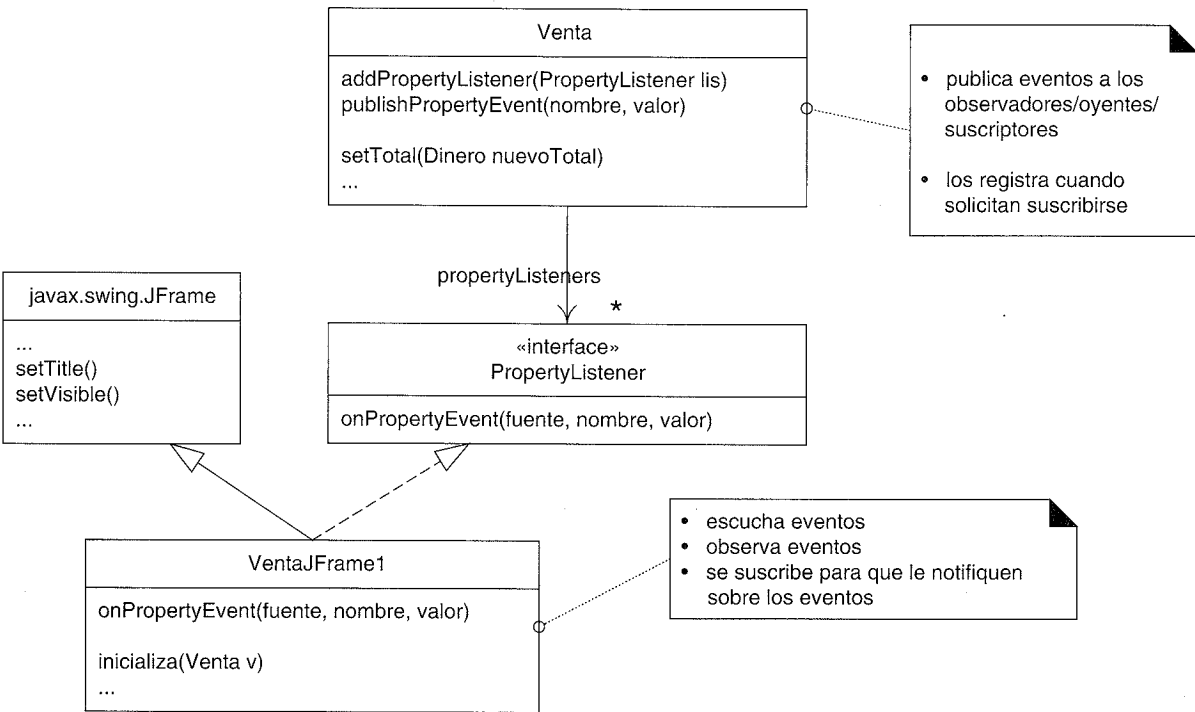


Figura 23.25. ¿Quién es el observador, oyente, suscriptor y emisor?

El Observador no sólo es para conectar UIs y objetos del modelo

El ejemplo anterior ilustraba la conexión con el Observador entre los objetos que no pertenecen a la UI y los objetos de la UI. Sin embargo, existen otros usos habituales.

El uso más común de ese patrón es para el manejo de los eventos de los elementos gráficos de la GUI, tanto en la tecnología Java (AWT y Swing) como en la .NET de Microsoft. Cada elemento gráfico es un emisor de eventos relacionados con la GUI, y otros objetos pueden suscribir su interés en ellos. Por ejemplo, un *JButton* de las Swing publica un “evento de acción” cuando se presiona. Otro objeto se registrará con el botón de manera que cuando se presione, se envía un mensaje al objeto y puede realizar alguna acción.

Otro ejemplo se muestra en la Figura 23.26 donde se ilustra un *RelojAlarma*, que es un emisor de eventos de alarma, y varios suscriptores. Este ejemplo es ilustrativo ya que pone de relieve que la interfaz *AlarmaListener* puede ser implementada por muchas clases, se pueden registrar simultáneamente muchos objetos como oyentes, y cada uno reaccionará ante el “evento de alarma” de manera diferente.

Un emisor puede tener muchos suscriptores a un evento

Como se desprende de la Figura 23.26, una instancia de emisor podría tener de uno a muchos suscriptores registrados. Por ejemplo, una instancia de *RelojAlarma* podría tener registrados tres objetos *VentanaDeAlarma*, cuatro objetos *Busca* y un *GuardianDeFiabilidad*. Cuando ocurre un evento de alarma, se le notifica a los ocho objetos *AlarmaListener* por medio del evento *onAlarmaEvent*.

Implementación

Eventos

Tanto en las implementaciones del Observador en Java como en C# .NET, se comunica un “evento” por medio de un mensaje ordinario, como *onPropertyEvent*. Además, en ambos casos, el evento se define más formalmente como una clase, y se rellena con los datos del evento apropiados. Entonces el evento se pasa como parámetro en el mensaje de evento.

Por ejemplo:

```
class PropertyEvent extends Event
{
    private Object fuenteDelEvento;
    private String nombrePropiedad;
    private Object valorAntiguo;
    private Object valorNuevo;
    //...
}

//...
```

```
class Venta
{
    private void publishPropertyEvent(
        String nombre, Object antiguo, Object nuevo)
    {
        PropertyEvent evt =
            new PropertyEvent (this, "venta.total", antiguo, nuevo);

        for each AlarmaListener al in alarmaListeners
            al.onPropertyEvent(evt);
    }

    //...
}
```

Java

Cuando se lanzó el JDK 1.0 en enero de 1996, contenía una implementación simple de publicar-suscribir basada en una clase y una interfaz denominadas *Observable* y

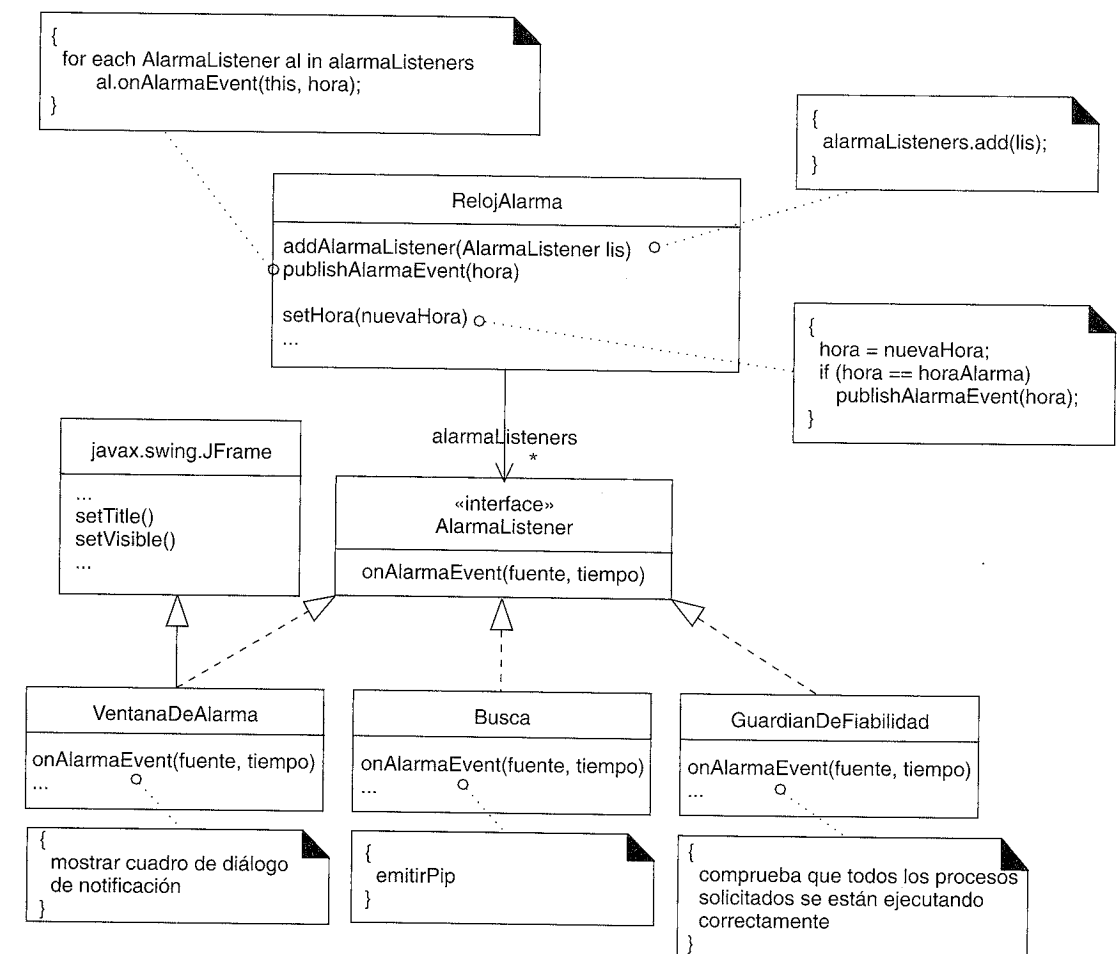


Figura 23.26. El Observador aplicado a los eventos de la alarma, con diferentes suscriptores.

Observer, respectivamente. Esto se copió básicamente, sin ninguna mejora, del enfoque de primeros de los ochenta de la implementación de publicar-suscribir en Smalltalk.

Por tanto, a finales de 1996, dentro de la versión JDK 1.1, se sustituyó el diseño Observable-Observer por el Modelo de Delegación de Eventos de Java (DEM, *Delegation Event Model*), una versión más robusta de publicar-suscribir, aunque se mantuvo el diseño original por compatibilidad con las versiones anteriores (pero en general debe evitarse).

Los diseños descritos en este capítulo son consistentes con el DEM, pero se han simplificado ligeramente para resaltar las ideas fundamentales.

Resumen

El Observador proporciona un modo de acoplar débilmente los objetos en términos de la comunicación. Los emisores conocen a los suscriptores sólo a través de una interfaz, y los suscriptores pueden registrarse (o darse de baja) de los emisores dinámicamente.

Patrones Relacionados

El Observador se basa en el Polimorfismo, y proporciona Variaciones Protegidas en cuanto a que protege al emisor del conocimiento de la clase específica de objetos, y número de objetos, con los que se comunica cuando genera un evento.

23.10. Conclusión

La lección principal que se puede extraer de esta exposición es que se pueden diseñar los objetos y asignar las responsabilidades con la ayuda de los patrones. Esto proporciona un conjunto explicable de estilos mediante los cuales se pueden construir sistemas orientados a objetos bien diseñados.

23.11. Lecturas adicionales

El libro de texto básico es *Design Patterns* de Gamma, Helm, Johnson y Vlissides, y es fundamental que lo lean todos los diseñadores de objetos.

Cada año se celebra un congreso "Pattern Languages of Programs" (PLOP, Lenguajes de Patrones de Programas) a partir del cual se publica una recopilación de patrones, en la serie *Pattern Languages of Program Design*, volumen 1, 2, etcétera. Se recomienda toda la serie.

Pattern-Oriented Software Architecture, volumen 1 y 2, promueve la discusión sobre los patrones en cuestiones de arquitectura a gran escala. El volumen 1 presenta una taxonomía de patrones.

Hay publicados cientos de patrones. Rising resume un porcentaje respetable de ellos en *The Pattern Almanac*.

PARTE 5 ELABORACIÓN EN LA ITERACIÓN 3

LA ITERACIÓN 3 Y SUS REQUISITOS

24.1. Requisitos de la Iteración 3

- Cuando no se puede acceder a los servicios remotos proporcionar el mantenimiento de los servicios ante los fallos mediante servicios locales. Por ejemplo, si no se puede acceder al producto remoto de la base de datos, utilice una versión local de datos almacenados en la caché.
- Proporcionar el soporte para el manejo de los dispositivos de PDV, como el cajón de caja y el dispensador de monedas.
- Manejar las autorizaciones de pago a crédito.
- Soporte para objetos persistentes.

24.2. Énfasis de la Iteración 3

Durante la fase de inicio y la Iteración 1 se exploraron una variedad de cuestiones fundamentales en el análisis de requisitos y el A/DOO. La Iteración 2 se ocupó en detalle del diseño de objetos. Esta tercera iteración considera de nuevo una perspectiva más amplia, explorando una extensa variedad de temas relacionados con el análisis y diseño, entre los que se encuentran:

- Relaciones entre casos de uso.
- Generalización y especialización.
- Modelado de estados.
- Arquitecturas en capas.
- El diseño de paquetes.
- Análisis arquitectural.
- Más patrones de diseño GoF.
- El diseño de *frameworks*, en particular un framework de persistencia.

RELACIONES ENTRE CASOS DE USO

¿Por qué los programadores confunden Halloween y las navidades? Porque OCT(31)=DEC(25).

Objetivos

- Relacionar los casos de uso mediante las asociaciones *include* y *extend*.
-

Introducción

Los casos de uso se pueden relacionar entre ellos. Por ejemplo, un caso de uso de subfunción como *Gestionar Pagos a Crédito* podría formar parte de varios casos de uso ordinarios, como *Procesar Venta* o *Procesar Alquiler*. La organización de los casos de uso mediante relaciones no influye en el comportamiento o los requisitos del sistema. Más bien, es simplemente una forma de organizar para (idealmente) mejorar la comunicación y la comprensión de los casos de uso, reducir la duplicación de texto y mejorar la gestión de los documentos de casos de uso.

Una advertencia

En algunas organizaciones que trabajan con casos de uso, se dedica mucho tiempo improductivo a debatir sobre cómo relacionar los casos de uso en los diagramas de casos de uso, en lugar de dedicarlo al trabajo importante de los casos de uso: escribir texto. En consecuencia, aunque este capítulo presenta las relaciones entre los casos de uso, el tema y la dedicación que merece deberían ponerse en perspectiva: tiene algo de valor, pero el trabajo importante es escribir el texto de los casos de uso. La especificación de los requisitos se hace escribiendo, no organizando los casos de uso, que es un paso opcional, posiblemente para ayudar a comprenderlos o reducir duplicaciones. Si un equipo co-

mienza el modelado de los casos de uso dedicando horas (o peor, días) discutiendo un diagrama de caso de uso y las relaciones entre los casos de uso (“¿Ésa debería ser una relación *include* o *extend*? ¿Deberíamos especializar este caso de uso?”), en lugar de centrarse en escribir rápidamente el texto clave del caso de uso, se está haciendo un esfuerzo significativo en el sitio equivocado.

Es más, la organización de los casos de uso utilizando las relaciones puede evolucionar iterativamente en pequeñas etapas a lo largo de la fase de elaboración; no es útil intentar aplicar un esfuerzo del estilo del modelo en cascada en el que se define y refina totalmente un diagrama completo de caso de uso y el conjunto de relaciones en una etapa al comienzo del proyecto.

25.1. La relación de inclusión (*include*)

Ésta es la relación más común e importante.

Es habitual tener algún comportamiento parcial común a varios casos de uso. Por ejemplo, la descripción del pago a crédito tiene lugar en varios casos de uso, entre los que se encuentran *Procesar Venta*, *Procesar Alquiler*, *Contribuir a Plan de Ahorro*, etcétera. En lugar de duplicar este texto, es conveniente separarlo en su propio caso de uso de subfunción, e indicar su inclusión. Esto es sencillamente factorizar y enlazar texto para evitar duplicaciones¹.

Por ejemplo:

UC1: Procesar Venta

...
Escenario principal de éxito:
1. El Cliente llega a un terminal PDV con mercancías y/o servicios para comprar.
...
7. El Cliente paga y el Sistema gestiona el pago.
...
Extensiones:
7b. Pago a crédito: Incluye Gestionar Pago a Crédito.
7c. Pago con cheque: Incluye Gestionar Pago con Cheque.
...

UC7: Procesar Alquiler

...
Extensiones:
6b. Pago a crédito: Incluye Gestionar Pago a Crédito.
...

¹ Sirve de ayuda que los enlaces se implementen también con hipervínculos navegables.

UC12: Gestionar Pago a Crédito

...
Nivel: Subfunción.
Escenario principal de éxito:
1. El Cliente introduce la información acerca de su cuenta de crédito.
2. El Sistema envía la solicitud de autorización del pago a un Sistema para el Servicio de Autorización de Pago, y solicita la aprobación del pago.
3. El Sistema recibe la aprobación del pago y la notifica al Cajero.
4. ...
Extensiones:
2a. El sistema detecta algún fallo al colaborar con el sistema externo:
1. El Sistema informa del error al Cajero.
2. El Cajero le pide al Cliente un modo de pago alternativo.
...

Ésta es la relación de inclusión (*include*).

Una notación algo más breve (y, por tanto, quizás se prefiera) para indicar el caso de uso que se incluye es simplemente subrayarlo o destacarlo de alguna manera. Por ejemplo:

UC1: Procesar Venta

...
Extensiones:
7b. Pago a crédito: Gestionar Pago a Crédito.
7c. Pago con cheque: Gestionar Pago con Cheque.

Nótese que el caso de uso de subfunción *Gestionar Pago a Crédito* se encontraba originalmente en la sección de *Extensiones* del caso de uso *Procesar Venta*, pero se factorizó aparte para evitar duplicaciones. También observe que en el caso de uso de subfunción se utilizan las mismas estructuras *Escenario principal de éxito* y *Extensiones* que en los casos de uso de procesos de negocio ordinarios como *Procesar Venta*.

Fowler ofrece una guía sencilla y práctica sobre cuándo utilizar la relación de inclusión [FS00]:

Utilice *include* cuando se está repitiendo en dos o más casos de usos separados y quiere evitar repeticiones.

Otro motivo es simplemente descomponer un caso de uso abrumadoramente largo en subunidades para mejorar la comprensión.

Utilización de include con el manejo de eventos asíncronos

La relación de inclusión también se utiliza para describir el manejo de un evento asíncrono, como cuando un usuario es capaz de seleccionar o bifurcar, en cualquier momento, a una ventana, función o página web específica, o en un rango de pasos.

De hecho, la notación de los casos de uso para representar esta bifurcación asíncrona ya se estudió en la introducción a los casos de uso en el Capítulo 6, pero en ese momento no se presentó la incorporación de una invocación a un subcaso de uso que se incluye.

La notación básica es utilizar etiquetas siguiendo el estilo *a**, *b**... en la sección de Extensiones. Recordemos que esto implica una extensión o evento que puede ocurrir en cualquier momento. Una variación menor es utilizar una etiqueta con un rango, como 3-9, cuando el evento asíncrono puede ocurrir en un rango relativamente amplio de los pasos del caso de uso, pero no todos.

UC1: Procesar Algo

...
Escenario principal de éxito:
1. ...
Extensiones:
a*. En cualquier momento, el Cliente selecciona la edición de la información personal: *Editar Información Personal*
b*. En cualquier momento, el Cliente selecciona imprimir la ayuda: *Presentar Ayuda Impresa*
2-11. El Cliente cancela: *Cancelar Confirmación de Transacción*
...

Resumen

La relación de inclusión se puede utilizar para la mayoría de problemas de relaciones de casos de uso. En resumen:

Factorice casos de uso de subfunción separados y utilice la relación *include* cuando:

- Están duplicados en otros casos de uso.
- Un caso de uso es *muy* complejo y largo, y separarlos en subunidades facilita la comprensión.

Como explicaremos, existen otras relaciones: extensión y generalización. Pero Cockburn, un modelador de casos de uso experto, aconseja escoger la relación de inclusión por encima de las relaciones de extensión y generalización:

Como primera regla empírica, utilice siempre la relación *include* entre los casos de uso. La gente que sigue esta regla declara que existen menos malentendidos entre ellos y sus lectores sobre lo que escriben que la gente que mezcla *include* con *extend* y *generalizes* [Cockburn01].

25.2. Terminología: casos de uso concretos, abstractos, base y adicional

Un **caso de uso concreto** es iniciado por un actor y lleva a cabo todo el comportamiento que desea el actor [RUP]. Éstos son los casos de uso de los procesos del negocio ele-

mentales. Por ejemplo, *Procesar Venta* es un caso de uso concreto. En cambio, un **caso de uso abstracto** nunca se instancia por él mismo; es un caso de uso de subfunción que forma parte de otro caso de uso. *Gestionar Pago a Crédito* es abstracto; no se mantiene independiente, sino que siempre forma parte de otra historia, como *Procesar Venta*.

Un caso de uso que incluye otro caso de uso, o que es extendido o especializado por otro caso de uso se denomina **caso de uso base**. *Procesar Venta* es un caso de uso base con respecto al caso de uso de subfunción *Gestionar Pago a Crédito* que incluye. Por otro lado, el caso de uso que es una inclusión, extensión o especialización se denomina **caso de uso adicional**. *Gestionar Pago a Crédito* es el caso de uso adicional en la relación de inclusión de *Procesar Venta*. Los casos de uso adicionales normalmente son abstractos. Los casos de uso base generalmente son concretos.

25.3. La relación de extensión (*extend*)

Suponga que el texto de un caso de uso no debiera modificarse (al menos no de manera significativa) por alguna razón. Quizás modificar continuamente el caso de uso con innumerables nuevas extensiones y pasos condicionales es un dolor de cabeza de mantenimiento, o se ha establecido el caso de uso como un artefacto estable, y no se puede tocar. ¿Cómo añadir al caso de uso sin modificar su texto original?

La relación de **extensión** (*extend*) proporciona una respuesta. La idea es crear un caso de uso que extiende o añade, y con él, describe dónde y bajo qué condiciones extiende el comportamiento de algún caso de uso base. Por ejemplo:

UC1: Procesar Venta (el caso de uso base)

...
Puntos de Extensión: *Cliente VIP*, paso 1. *Pago*, paso 7.
Escenario principal de éxito:
1. El Cliente llega a un terminal PDV con mercancías y/o servicios para comprar.
...
7. El Cliente paga y el Sistema gestiona el pago.
...

UC15: Gestionar Pago con Vale-Regalo (el caso de uso que extiende)

...
Activa: El Cliente quiere pagar con vale-regalo
Puntos de Extensión: Pago en Procesar Venta
Nivel: Subfunción.
Escenario Principal de Éxito:
1. El Cliente le da el vale-regalo al Cajero.
2. El Cajero introduce el identificador del vale-regalo.
...

Éste es un ejemplo de una relación de **extensión**. Observe el uso de un **punto de extensión**, y que el caso de uso que extiende se activa por alguna condición. Los puntos de extensión son etiquetas en el caso de uso base que extiende referencia como puntos de extensión, de manera que los números de los pasos del caso de uso base pueden cambiar sin afectar al caso de uso que extiende —indirección una vez más—.

Algunas veces, el punto de extensión es simplemente “En cualquier punto del caso de uso X”. Esto es especialmente habitual en sistemas con muchos eventos asíncronos, como un procesador de texto (“Hacer ahora la corrección ortográfica”, “Hacer ahora una búsqueda en el tesauro”), o sistemas de control reactivo. Observe sin embargo, como se describe en la sección anterior de la relación de inclusión, que también se puede utilizar la relación de inclusión para describir el manejo de eventos asíncronos. La alternativa *extend* es una opción cuando el caso de uso base está cerrado a las modificaciones.

Observe que una señal de calidad de la relación de extensión es que el caso de uso base (*Procesar Venta*) no referencia al caso de uso que lo extiende (*Gestionar Pago con Vale-Regalo*) y, por tanto, no define o controla las condiciones bajo las cuales se activan las extensiones. El caso de uso *Procesar Venta* es completo y forma un todo por él mismo, sin conocer los casos de uso que lo extiende.

Fíjese que el caso de uso adicional *Gestionar Pago con Vale-Regalo* de manera alternativa se podría haber referenciado en *Procesar Venta* mediante una relación de inclusión, como con *Gestionar Pago a Crédito*. Con frecuencia es adecuado. Pero este ejemplo estaba motivado por la restricción de que el caso de uso *Procesar Venta* no se iba a modificar, que es la situación en la que se utiliza la extensión en lugar de la inclusión.

Además, nótese que este escenario del vale-regalo se podría haber registrado simplemente añadiéndolo como una extensión en la sección de *Extensiones* de *Procesar Venta*. Este enfoque evita tanto la relación de inclusión como la de extensión, y la creación de un caso de uso de subfunción separado.

De hecho, normalmente es preferible actualizar simplemente la sección de *Extensiones*, en lugar de crear complejas relaciones de casos de uso.

Algunas guías de casos de uso recomiendan la utilización de casos de uso que extienden y la relación de extensión para modelar el comportamiento condicional u opcional del caso de uso base. Esto no es incorrecto, pero no comprende que el comportamiento condicional u opcional se puede registrar simplemente como texto en la sección de *Extensiones* del caso de uso base. La dificultad de utilizar la relación de extensión y más casos de uso no está motivada solamente por el comportamiento opcional.

Lo que motiva esencialmente el uso de la técnica de extensión es cuando no es conveniente por alguna razón modificar el caso de uso base.

25.4. La relación de generalización (*generalize*)

La discusión acerca de la relación de generalización queda fuera del alcance de este libro. Sin embargo, observe que los expertos en casos de uso han estado realizando con éxito el trabajo sobre casos de uso sin esta relación opcional, que añade otro nivel de complejidad a los casos de uso, y todavía no existe un acuerdo entre los que la utilizan

sobre una guía de buenas prácticas de cómo sacar provecho de esta idea. Los consultores sobre casos de uso observan que habitualmente da lugar a complicaciones y que se dedica mucho tiempo improductivo a la inclusión de muchas relaciones de casos de uso.

25.5. Diagramas de casos de uso

La Figura 25.1 ilustra la notación UML para la relación de inclusión, que es la única que se va a utilizar en el caso de estudio, siguiendo el consejo de los expertos en casos de uso de mantener las cosas sencillas y preferir la relación de inclusión.

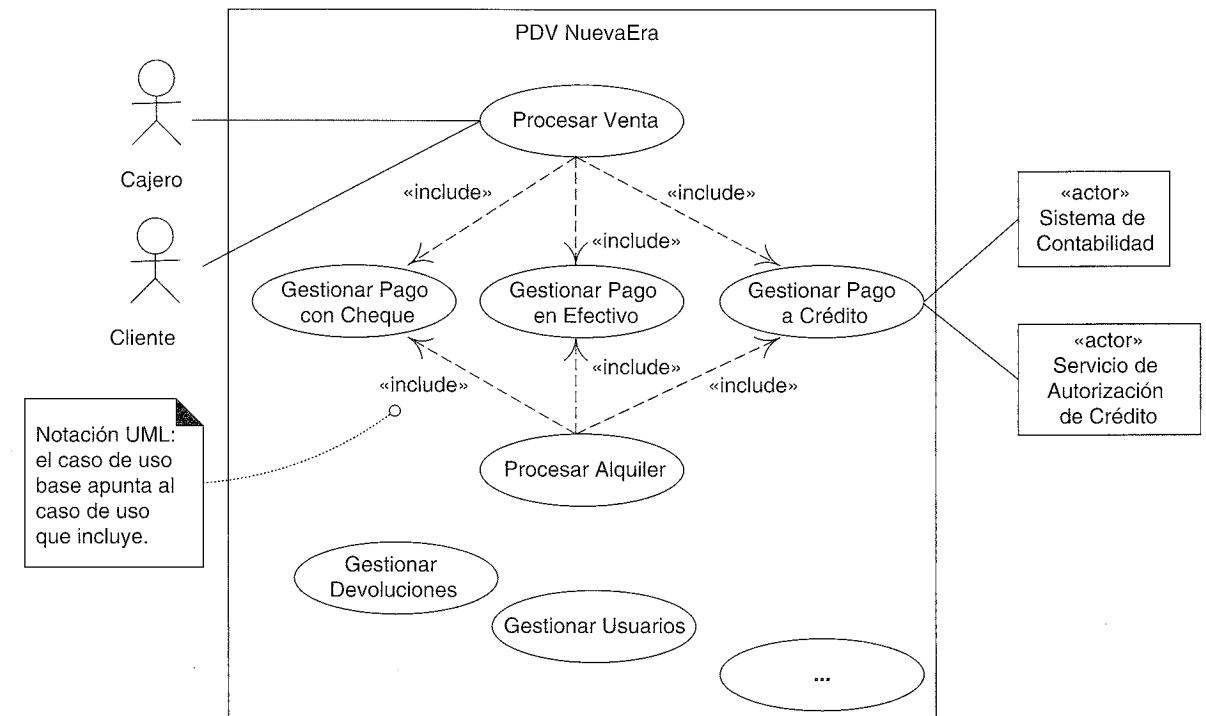


Figura 25.1. Relación de inclusión de los casos de uso en el Modelo de Casos de Uso.

La notación de la relación de extensión se ilustra en la Figura 25.2.

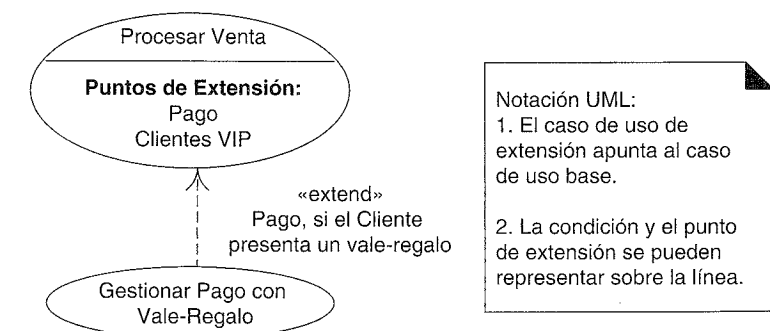


Figura 25.2. La relación de extensión.



MODELADO DE LA GENERALIZACIÓN

Las clasificaciones simplonas y las generalizaciones falsas son las maldiciones de la vida organizada.

Una generalización de H. G. Wells

Objetivos

- Crear jerarquías de generalización-especialización.
 - Identificar cuándo merece la pena mostrar una subclase.
 - Aplicar las pruebas del “100%” y “Es-un” para validar las subclases.
-

Introducción

La generalización y la especialización son conceptos fundamentales en el modelado del dominio que favorece una economía de palabras; más aún, las jerarquías de clases conceptuales a menudo constituyen la fuente de inspiración para las jerarquías de clases software que se aprovechan de la herencia y reducen la duplicación de código.

26.1. Nuevos conceptos para el Modelo del Dominio

Como en la iteración 1, el Modelo del Dominio del UP podría desarrollarse incrementalmente considerando los conceptos en los requisitos para esta iteración. Nos servirán de ayuda técnicas como la *Lista de Categorías de Conceptos* y la identificación de frases nominales. Un enfoque efectivo para desarrollar un modelo del dominio robusto y rico es estudiar el trabajo de otros autores sobre este tema, como [Fowler96]. Muchas de las cuestiones sutiles de modelado que ellos exploran quedan fuera del alcance de este libro.

Lista de Categorías de Conceptos

La Tabla 26.1 muestra algunos conceptos relevantes que se van a tener en cuenta en esta iteración.

Tabla 26.1. Lista de Categorías de Conceptos.

Categoría	Ejemplos
objetos físicos o tangibles	TarjetaDeCredito, Cheque
especificaciones, diseños o descripciones de cosas	
lugares	
transacciones	PagoEnEfectivo, PagoACredito, PagoConCheque
líneas de la transacción	
roles de la gente	
contenedores de otras cosas	
cosas en un contenedor	
otros sistemas informáticos o electro-mecánicos externos a nuestro sistema	ServicioAutorizacionCredito ServicioAutorizacionCheque
conceptos abstractos	
organizaciones	ServicioAutorizacionCredito ServicioAutorizacionCheque
eventos	
reglas y políticas	
catálogos	
registros de cuentas, trabajo, contratos, cuestiones legales	CuentasPorCobrar
instrumentos financieros y servicios	
manuales, libros	

Identificación de frases nominales

Recordemos que la identificación de frases nominales no se puede aplicar mecánicamente para identificar los conceptos relevantes que se van a incluir en el modelo del dominio. Se debe aplicar el sentido común y desarrollar las abstracciones adecuadas, puesto que el lenguaje natural es ambiguo y los conceptos relevantes no siempre se encuentran de manera explícita o clara en el texto existente. Sin embargo, es una técnica práctica en el modelado del dominio puesto que es directa.

Esta iteración maneja los escenarios del caso de uso *Procesar Venta* para los pagos a crédito y con cheque. A continuación se muestran algunas de las frases nominales identificadas a partir de estas extensiones:

Caso de Uso UC1: Procesar Venta

- ...
- Extensiones:**
- 7b. Pago a crédito:
- 1. El Cliente introduce la **información de su cuenta de crédito**.
 - 2. El Sistema envía la **petición de autorización del pago** al Sistema de **Servicio de Autorización de Pago** externo, y solicita la **aprobación del pago**.
 - 2a. El Sistema detecta un fallo en la colaboración con el sistema externo:
 - 1. El Sistema señala el error al Cajero.
 - 2. El Cajero le pide al Cliente un modo de pago alternativo.
 - 3. El Sistema recibe la **aprobación del pago** y lo notifica al Cajero.
 - 3a. El Sistema recibe la **denegación del pago**:
 - 1. El Sistema señala la denegación al Cajero.
 - 2. El Cajero le pide al Cliente un modo de pago alternativo.
 - 4. El Sistema registra el **pago a crédito**, que incluye la aprobación del pago.
 - 5. El Sistema presenta el mecanismo de entrada para la firma del pago a crédito.
 - 6. El Cajero le pide al Cliente que firme el pago a crédito. El Cliente introduce la firma.
 - 7c. Pago con cheque:
 - 1. El Cliente escribe un **cheque**, y lo da junto con su **carnet de conducir** al Cajero.
 - 2. El Cajero escribe el número del carnet de conducir en el cheque, lo introduce, y solicita la **autorización del pago con cheque**.
 - 3. Genera una **solicitud del pago con cheque** y lo envía al **Servicio de Autorización de Cheques**.
 - 4. Recibe la aprobación del pago con cheque y notifica la aprobación al Cajero.
 - 5. El sistema registra el **pago con cheque**, que incluye la aprobación del pago.
- ...

Transacciones del servicio de autorización

La identificación de frases nominales revela conceptos como *SolicitudPagoACredito* y *RespuestaAprobacionCredito*. Éstos en realidad podrían verse como transacciones de los servicios externos, y en general, resulta útil identificar tales transacciones porque las actividades y los procesos tienden a girar alrededor de ellos.

Estas transacciones no tienen que representar registros informáticos o bits que viajan por una línea. Representan la abstracción de la transacción independientemente del modo en el que se ejecute. Por ejemplo, la solicitud de un pago a crédito podría ejecutarse mediante una llamada telefónica o el envío de registros o mensajes entre dos ordenadores, etcétera.

26.2. Generalización

Los conceptos de *PagoEnEfectivo*, *PagoACredito*, y *PagoConCheque* son todos muy parecidos. En esta situación, es posible (y útil ¹) organizarlos (como en la Figura 26.1) en una **jerarquía de clases de generalización-especialización** (o simplemente **jerarquía**

¹ Más tarde, en este capítulo, estudiaremos los motivos para definir jerarquías de clases.

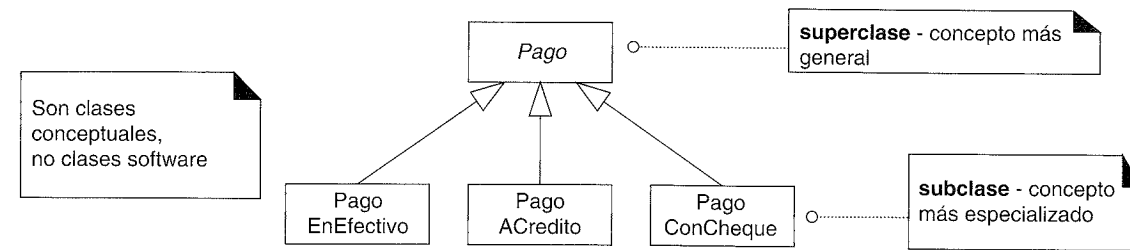


Figura 26.1. Jerarquía de generalización-especialización.

de clases) en la cual, la **superclase** *Pago* representa un concepto más general, y las **subclases** conceptos más especializados.

Nótese que la presentación de las clases en este capítulo se refiere a las clases *conceptuales* y no a las clases *software*.

La **generalización** es la actividad de identificar elementos comunes entre los conceptos y definir las relaciones de superclase (concepto general) y subclase (concepto especializado). Es una forma de construir clasificaciones taxonómicas entre los conceptos que entonces se representan en jerarquías de clases.

La identificación de una superclase y las subclases es útil en un modelo del dominio porque su presencia nos permite entender los conceptos en términos más generales, refinados y abstractos. Esto nos lleva a una economía de expresión, a mejorar la comprensión y a reducir la información repetida. Y aunque ahora nos estamos centrando en el Modelo del Dominio del UP y no en el Modelo de Diseño del software, el diseño e implementación posterior de la superclase y subclases, como clases software que utilizan la herencia, producirán un mejor software.

Por tanto:

Identifique las superclases y subclases del dominio relevantes para el estudio actual, y represéntelas en el Modelo del Dominio.

Notación UML: Revisemos la notación de la generalización que se introdujo en un capítulo anterior, en UML, la relación de generalización entre elementos se representa con una línea con un triángulo hueco grande en el extremo que apunta a la clase más general desde las más especializadas (ver Figura 26.2). Se puede utilizar tanto un estilo con líneas separadas o unidas.

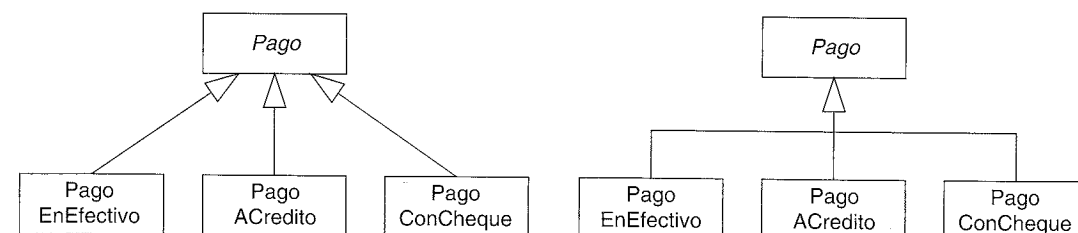


Figura 26.2. Jerarquía de clases con notaciones de líneas separadas o unidas.

26.3. Definición de superclases y subclases conceptuales

Puesto que es útil identificar las super- y subclases conceptuales, es conveniente entender con claridad y precisión la generalización, superclases y subclases en términos de la definición de clase y conjuntos de clases². De esto se ocupan las siguientes secciones.

Generalización y definición de clase conceptual

¿Cuál es la relación de una superclase conceptual con una subclase?

La definición de una superclase conceptual es más general y abarca más que la definición de una subclase.

Por ejemplo, considere la superclase *Pago* y sus subclases (*PagoEnEfectivo*, etcétera). Asuma que la definición del *Pago* representa la transacción de transferir dinero (no necesariamente en efectivo) para una compra desde una parte a otra, y que todos los pagos tienen una cantidad de dinero que es la que transfieren. El modelo que se corresponde a esta definición es el que se muestra en la Figura 26.3.

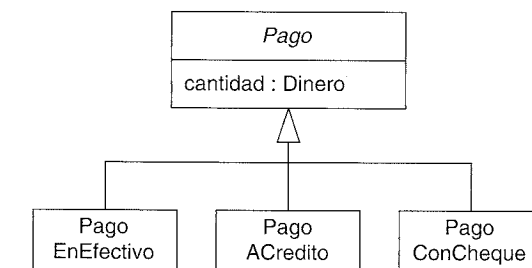


Figura 26.3. Jerarquía de clases del pago.

Un *PagoACredito* es una transferencia de dinero por medio de una institución de crédito que necesita que se autorice. Mi definición de *Pago* abarca más y es más general que mi definición de *PagoACredito*.

Generalización y conjuntos de clases

Las superclases y las subclases conceptuales están relacionadas en términos de pertenencia a un conjunto.

Todos los miembros del conjunto de una subclase conceptual son miembros del conjunto de su superclase.

² Es decir, la intensión y extensión de la clase. Esta discusión está inspirada en [MO95].

Por ejemplo, en términos de la pertenencia a un conjunto, todas las instancias del conjunto de *PagoACredito* también son miembros del conjunto de *Pago*. En un diagrama de Venn, se representa como en la Figura 26.4.

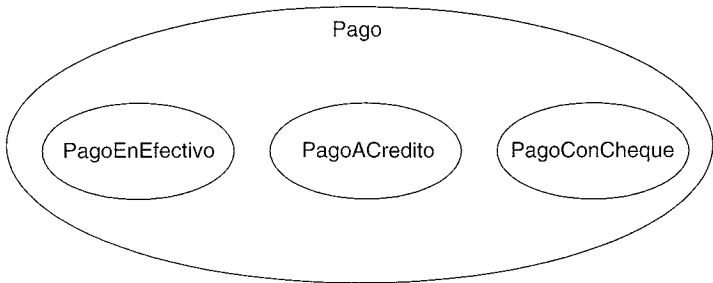


Figura 26.4. Diagrama de Venn de las relaciones de conjuntos.

Conformidad de la definición de la subclase conceptual

Cuando se crea una jerarquía de clases, se hacen declaraciones sobre las superclases que se aplican a las subclases. Por ejemplo, la Figura 26.5 establece que todos los *Pagos* tienen una *cantidad* y que se asocian con una *Venta*.

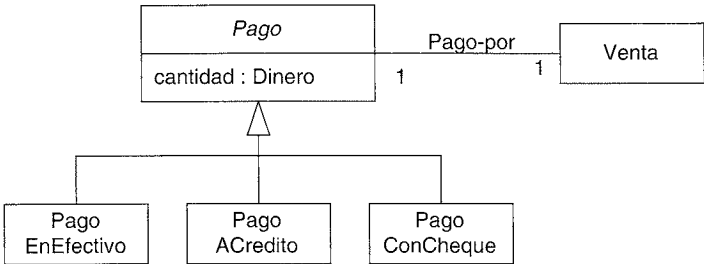


Figura 26.5. Conformidad de las subclases.

Todas las subclases de *Pago* deben ajustarse a tener una cantidad y ser el pago de una *Venta*. En general, esta regla de conformidad con la definición de una superclase es la *Regla del 100%*:

Regla del 100%

El 100% de la definición de la superclase conceptual se debe poder aplicar a la subclase. La subclase debe ajustarse al 100% de los:

- Atributos.
- Asociaciones de la superclase.

Conformidad del conjunto de la subclase conceptual

Una subclase conceptual debería ser un miembro del conjunto de la superclase. Por tanto, un *PagoACredito* debería ser un miembro del conjunto de los *Pagos*.

Informalmente, esto expresa la noción de que la subclase conceptual *es un tipo de* superclase. Un *PagoACredito es un tipo de Pago*. De manera más concisa, *es-un-tipo-de* se denomina *es-un*.

Este tipo de conformidad es la *Regla Es-un*:

Regla Es-un:

Todos los miembros del conjunto de una subclase deben ser miembros del conjunto de su superclase.

En lenguaje natural, esto puede comprobarse informalmente formando la sentencia: Subclase **es una** Superclase.

Por ejemplo, la sentencia *PagoACredito es un Pago* tiene sentido, y transmite la noción de conformidad con la pertenencia a un conjunto.

¿Qué es una subclase conceptual correcta?

A partir de la exposición anterior, aplique las siguientes pruebas³ para definir una subclase correcta cuando construya un modelo del dominio:

Una subclase potencial debería estar de acuerdo con:

- La Regla del 100% (conformidad en la definición).
- La Regla Es-un (conformidad con la pertenencia al conjunto).

26.4 Cuándo definir una clase conceptual

Se han presentado las reglas que aseguran que una subclase es correcta (las reglas del 100% y Es-un). Sin embargo, ¿cuándo deberíamos preocuparnos de definir una subclase? En primer lugar, una definición: Una **partición de clases conceptuales** es una división de las clases conceptuales en subclases disjuntas (o **tipos** según la terminología de Odell) [MO95].

La pregunta se podría volver a enunciarse como:

“¿Cuándo es útil representar una partición de clases conceptuales?”

Por ejemplo, en el dominio del PDV, el *Cliente* podría particionarse correctamente en (o dividirse en las subclases) *ClienteHombre* y *ClienteMujer*. Pero, ¿es relevante o útil mostrar esto en nuestro modelo (ver Figura 26.6)?

Esta partición no es útil para nuestro dominio; la siguiente sección explica por qué.

³ Se han elegido estos nombres de las clases por su ayuda nemotécnica en lugar de por precisión.

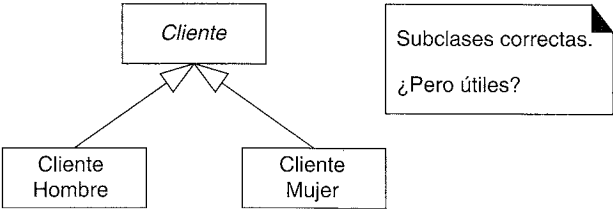


Figura 26.6. Partición de clases conceptuales legales, ¿pero es útil en nuestro dominio?

Razones para particionar una clase conceptual en subclases

A continuación presentamos razones de peso para particionar una clase en subclases:

Cree una subclase conceptual de una superclase cuando:

1.

La subclase tiene atributos adicionales de interés.

2.

La subclase tiene asociaciones adicionales de interés.

3.

El concepto de la subclase funciona, se maneja, reacciona, o se manipula de manera diferente a la superclase o a otras subclases, de alguna manera que es interesante.

4.

El concepto de la subclase representa una cosa animada (por ejemplo, animal o robot) que se comporta de manera diferente a la superclase o a otras subclases, de alguna manera que es interesante.

En base al criterio anterior, la partición del *Cliente* en las subclases *ClienteHombre* y *ClienteMujer* no está justificada porque no tienen atributos adicionales o asociaciones, y no se opera sobre ellos (se les trata) de manera diferente, y no se comportan de manera diferente de alguna manera que sea de interés⁴.

La Tabla 26.2 muestra algunos ejemplos de particiones de clases en el dominio de los pagos y otras áreas, utilizando este criterio.

Tabla 26.2. Ejemplo de particiones de subclases.	
Motivación de la subclase conceptual	Ejemplos
La subclase tiene atributos adicionales de interés	Pagos: no aplicable Biblioteca: <i>Libro</i> , subclase de <i>RecursoPrestable</i> , tiene un atributo <i>ISBN</i> .
La subclase tiene asociaciones adicionales de interés	Pagos: <i>PagoACredito</i> , subclase de <i>Pago</i> , asociada con una <i>TarjetaDeCredito</i> . Biblioteca: <i>Vídeo</i> , subclase de <i>RecursoPrestable</i> , asociada con un <i>Encargado</i> .

continúa

⁴ Los hombres y las mujeres tienen diferentes hábitos a la hora de comprar. Sin embargo, éstos no son relevantes para los requisitos del caso de uso actual —el criterio que delimita nuestro estudio—.

Tabla 26.2. Ejemplo de particiones de subclases. (Continuación)

Motivación de la subclase conceptual	Ejemplos
El concepto de la subclase funciona, se gestiona, reacciona, o se manipula de manera diferente a la superclase o a otras subclases, de alguna manera que es interesante.	Pagos: <i>PagoACredito</i> , subclase de <i>Pago</i> , se gestiona de manera diferente a otros tipos de pagos en el modo de autorizarlo. Biblioteca: <i>Software</i> , subclase de <i>RecursoPrestable</i> , requiere un depósito antes de que pueda prestarse.
El concepto de la subclase representa una cosa animada (por ejemplo, animal o robot) que se comporta de manera diferente a la superclase o a otras subclases, de alguna manera que es interesante.	Pagos: no aplicable. Biblioteca: no aplicable. Investigación de Mercado: <i>Hombre</i> , subclase de <i>Persona</i> , se comporta de manera diferente a la <i>Mujer</i> con respecto a los hábitos de compras.

26.5. Cuándo definir una superclase conceptual

Normalmente, se aconseja generalizar en una superclase común cuando se identifican elementos comunes entre subclases potenciales. A continuación presentamos los motivos para generalizar y definir una superclase:

Cree una superclase conceptual en una relación de generalización de subclases cuando:

•

Cuando las subclases potenciales representan variaciones de un concepto similar.

•

Las subclases se ajustarán a las reglas del 100% y Es-un.

•

Todas las subclases tienen el mismo atributo que se puede factorizar y expresar en la superclase.

•

Todas las subclases tienen la misma asociación que se puede factorizar y relacionar con la superclase.

Las siguientes secciones ilustran estos puntos.

26.6. Jerarquías de clases conceptuales del PDV NuevaEra

Clases de Pago

Si nos basamos en el criterio anterior para particionar la clase *Pago*, es útil crear una jerarquía de clases de varios tipos de pagos. La justificación de las clases y subclases se muestra en la Figura 26.7.

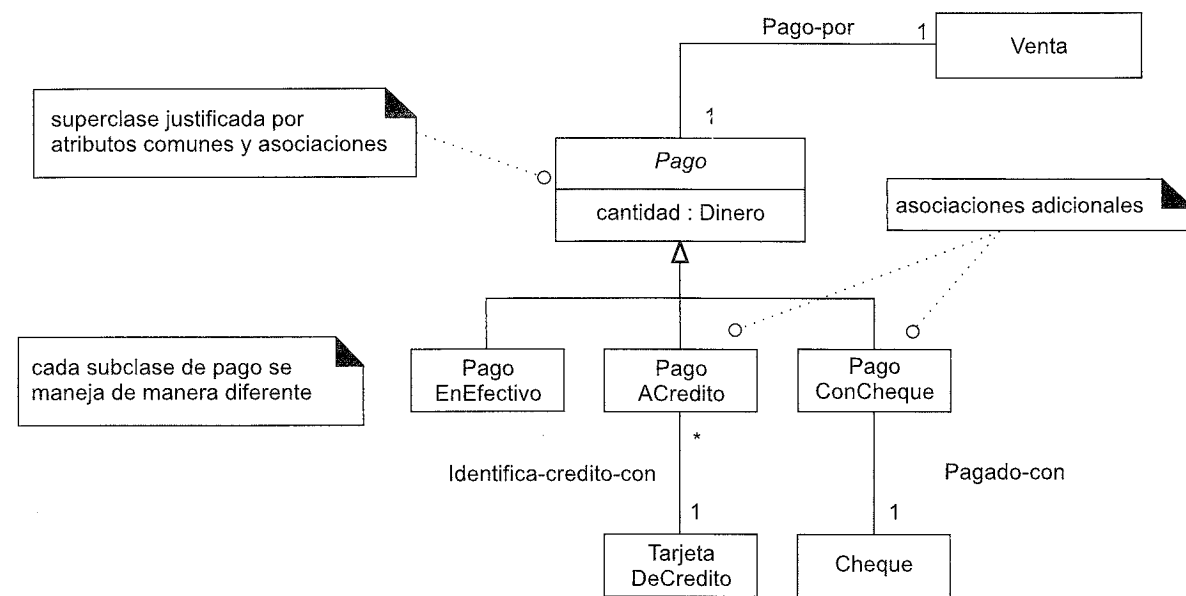


Figura 26.7. Justificación de las subclases de Pago.

Clases de Servicio de Autorización

Los servicios de autorización de crédito y cheques son variaciones de un concepto similar, y tienen atributos comunes de interés. Esto nos lleva a la jerarquía de clases de la Figura 26.8.

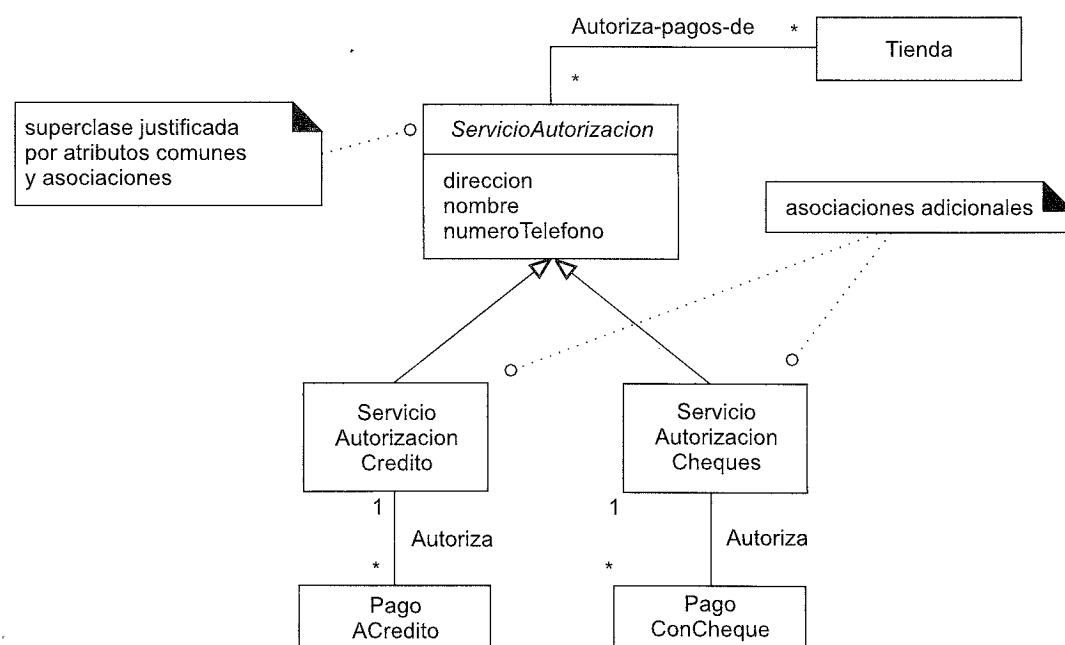


Figura 26.8. Justificación de la jerarquía de ServicioAutorización.

Clases de Transacción de Autorización

El modelado de distintos tipos de transacciones del servicio de autorización (solicitudes y respuestas) presenta un caso interesante. En general, es útil mostrar las transacciones de los servicios externos en un modelo del dominio porque las actividades y los procesos suelen girar alrededor de ellas. Son conceptos importantes.

¿Debería el modelador representar *todas* las variaciones de una transacción de un servicio externo? Depende. Como se mencionó, los modelos del dominio no son necesariamente correctos o incorrectos, sino más bien, son más o menos útiles. Son útiles, porque cada clase de transacción está relacionada con diferentes conceptos, procesos y reglas del negocio⁵.

Una segunda pregunta interesante es el grado de generalización que es conveniente mostrar en un modelo. Por motivos de la explicación, asumamos que cada clase transacción tiene una fecha y hora. Estos atributos comunes, junto con el deseo de crear una generalización final para esta familia de conceptos relacionados, justifica la creación de *TransaccionAutorizacionPago*.

¿Pero es útil generalizar una respuesta en *RespuestaAutorizacionPagoACredito* y *RespuestaAutorizacionPagoConCheque*, como se muestra en la Figura 26.9, o es suficiente menos generalización, como se muestra en la Figura 26.10?

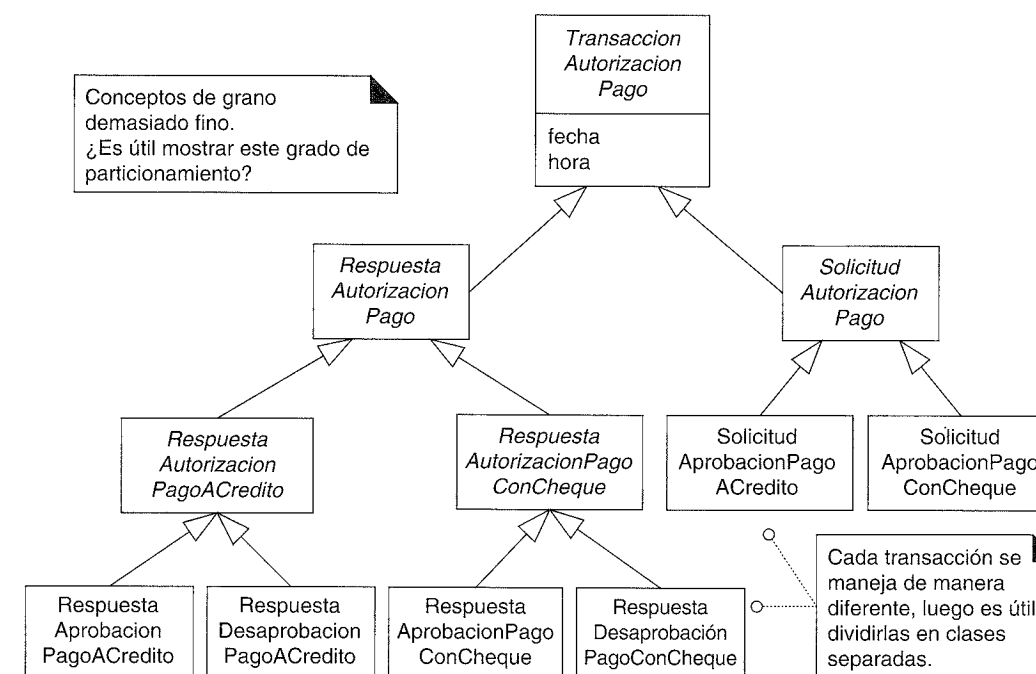


Figura 26.9. Una posible jerarquía de clases para las transacciones con los servicios externos.

⁵ En los modelos del dominio de las telecomunicaciones, es igualmente útil identificar cada tipo de mensaje de intercambio o cambio.

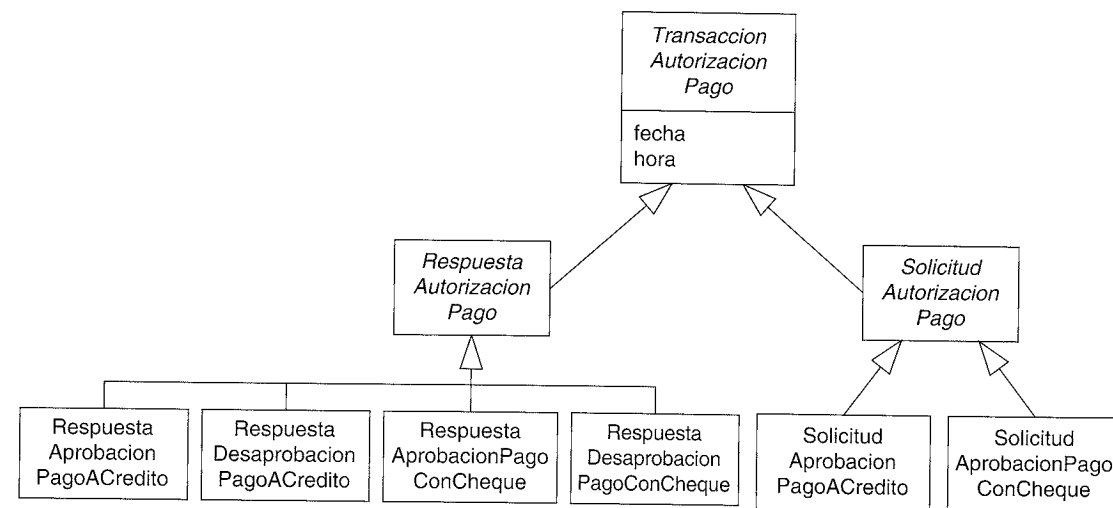


Figura 26.10. Una jerarquía de clases de transacción alternativa.

La jerarquía de clases que se muestra en la Figura 26.10 es suficientemente útil en cuanto a la generalización, porque las generalizaciones adicionales no añaden ningún valor obvio. La jerarquía de la Figura 26.9 expresa una granularidad de generalización más fina que no enriquece de manera significativa nuestra comprensión de los conceptos y reglas del negocio, sino que hace el modelo más complejo —y no es conveniente añadir complejidad a menos que proporcione otros beneficios.

26.7. Clases conceptuales abstractas

Es útil identificar las clases abstractas en el modelo del dominio porque restringe las clases que pueden tener instancias concretas; por tanto, clarifica las reglas del dominio del problema.

Si cada miembro de una clase C debe ser también un miembro de una subclase, entonces la clase C se denomina **clase conceptual abstracta**.

Por ejemplo, asuma que cada instancia de *Pago* deber ser más concretamente una instancia de la subclase *PagoACredito*, *PagoEnEfectivo* o *PagoConCheque*. Esto se ilustra en el diagrama de Venn de la Figura 26.11 (b). Puesto que cada miembro de *Pago* también es un miembro de una de las subclases, el *Pago* es una clase conceptual por definición.

En cambio, si puede haber instancias de *Pago* que no son miembros de una subclase, no es una clase abstracta, como se ilustra en la Figura 26.11 (a).

En el dominio del PDV, cada *Pago* es realmente un miembro de una subclase. La Figura 26.11 (b) es la descripción correcta de los pagos, el *Pago* es una clase conceptual abstracta.

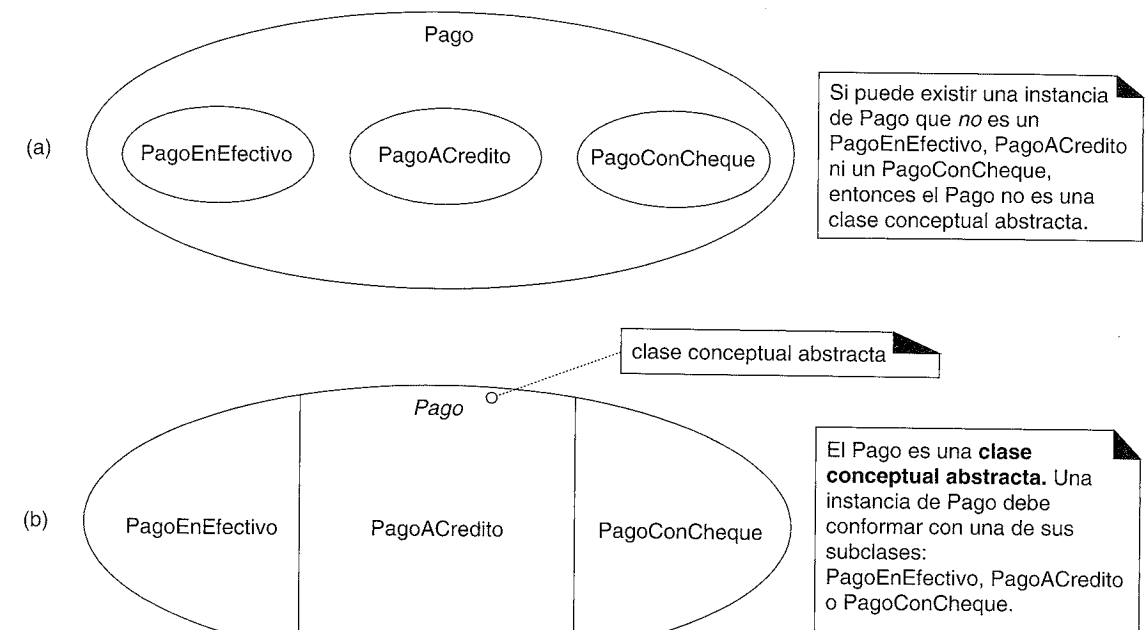


Figura 26.11. Clases conceptuales abstractas.

Notación para las clases abstractas en UML

Recordemos que UML proporciona una notación para representar las clases abstractas —el nombre de la clase en cursiva (ver Figura 26.12)—.

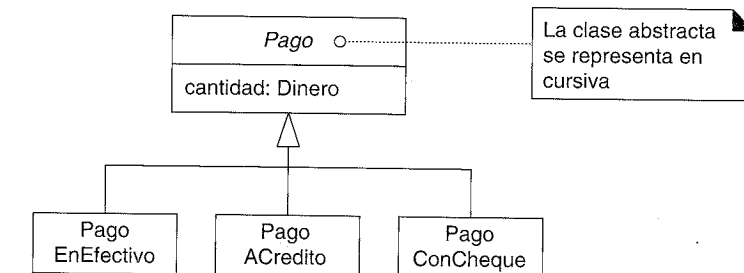


Figura 26.12. Notación para las clases abstractas.

Identifique las clases abstractas y represéntelas con un nombre en cursiva en el Modelo del Dominio.

26.8. Modelado de los cambios de estado

Asuma que un pago puede estar o en estado autorizado o no autorizado, y es significativo mostrarlo en el modelo del dominio (podría no serlo realmente, pero asímalo para la exposición). Como se muestra en la Figura 26.13, un enfoque de modelado es definir

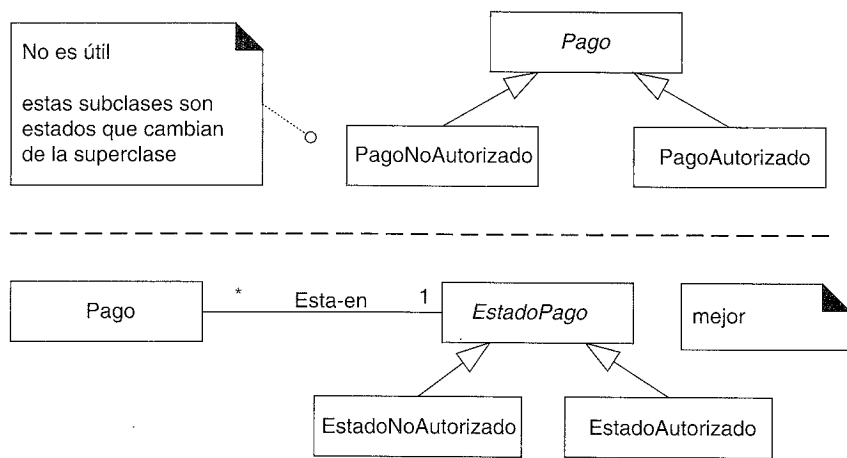


Figura 26.13. Modelado de los cambios de estado.

subclases de *Pago*: *PagoNoAutorizado* y *PagoAutorizado*. Sin embargo, observe que un pago no permanece en uno de estos estados; normalmente cambia de no autorizado a autorizado. Esto nos lleva a la siguiente guía:

No modele el estado de un concepto X como subclases de X. Más bien, o:

- Defina una jerarquía de estados y asocie los estados con X, o
- Ignore la representación de los estados de un concepto en el modelo del dominio; en lugar de eso represente los estados en diagramas de estados.

26.9. Jerarquías de clases y herencia en el software

La explicación de las jerarquías de clases conceptuales no ha mencionado la *herencia* porque la discusión se centra en un modelo del dominio de cosas del mundo, no en artefactos software. En un lenguaje de programación orientado a objetos, una subclase software **hereda** la definición de los atributos y operaciones de sus superclases mediante la creación de **jerarquías de clases software**. La **herencia** es un mecanismo software para hacer que las cosas de la superclase se apliquen a las subclases. Permite factorizar código de las subclases y subirlo arriba de la jerarquía de clases. Por tanto, la herencia no tiene ningún papel real que desempeñar en la discusión del modelo del dominio, aunque lo tiene cuando pasamos al punto de vista del diseño o la implementación.

Las jerarquías de clases conceptuales generadas aquí podrían reflejarse, o no, en el Modelo de Diseño. Por ejemplo, la jerarquía de clases de transacciones de los servicios externos podría reunirse o expandirse en jerarquías de clases alternativas, dependiendo de las características del lenguaje y otros factores. Por ejemplo, las clases plantilla (*template*) de C++ algunas veces pueden reducir el número de clases.

REFINAMIENTO DEL MODELO DEL DOMINIO

PRESENTE, n. Aquella parte de la eternidad que divide el dominio de la desilusión del reino de la esperanza.
Ambrose Bierce.

Objetivos

- Añadir clases asociación al Modelo del Dominio.
- Añadir relaciones de agregación.
- Modelar los intervalos de tiempo de la información donde es aplicable.
- Elegir cómo modelar roles.
- Organizar el Modelo del Dominio en paquetes.

Introducción

Este capítulo presenta ideas útiles y notación adicional disponible para el modelado del dominio, y las aplica para refinar aspectos del Modelo del Dominio del PDV NuevaEra.

27.1 Clases asociación

Los siguientes requisitos del dominio disponen el escenario para las clases asociación:

- Los servicios de autorización asignan un ID de comerciante a cada tienda para que se identifique en las comunicaciones.
- Una solicitud de autorización de pago desde la tienda a un servicio de autorización necesita el ID de comerciante que identifica la tienda al servicio.

- Además, una tienda tiene un ID de comerciante distinto para cada servicio.
¿Dónde debería residir el ID de comerciante en el Modelo del Dominio del UP?

Es incorrecto colocar el *comercianteID* en la *Tienda* porque una *Tienda* puede tener más de un valor para el *comercianteID*. Lo mismo ocurre si lo colocamos en el *ServicioAutorizacion* (ver Figura 27.1).

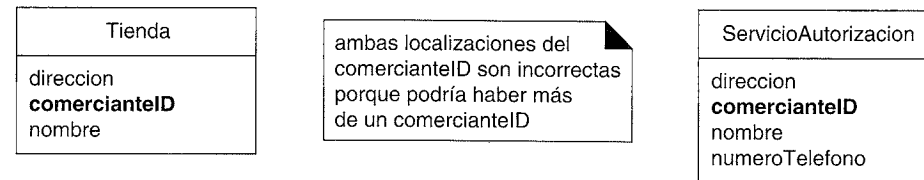


Figura 27.1. Uso inapropiado de un atributo.

Esto nos lleva al siguiente principio de modelado:

En un modelo del dominio, si una clase C puede tener simultáneamente muchos valores para el mismo tipo de atributo A, no coloque el atributo A en C. Coloque el atributo A en otra clase que esté asociada con C.

Por ejemplo:

- Una *Persona* podría tener muchos números de teléfono. Coloque el número de teléfono en otra clase, como *NumeroTelefono* o *InformacionDeContacto*, y asocie muchas de éstas a *Persona*.

El principio anterior sugiere que es más apropiado un modelo como el de la Figura 27.2. En el mundo de los negocios, ¿qué concepto recoge formalmente la información relacionada con los servicios que un servicio proporciona a un cliente? —un *Contrato* o *Cuenta*—.

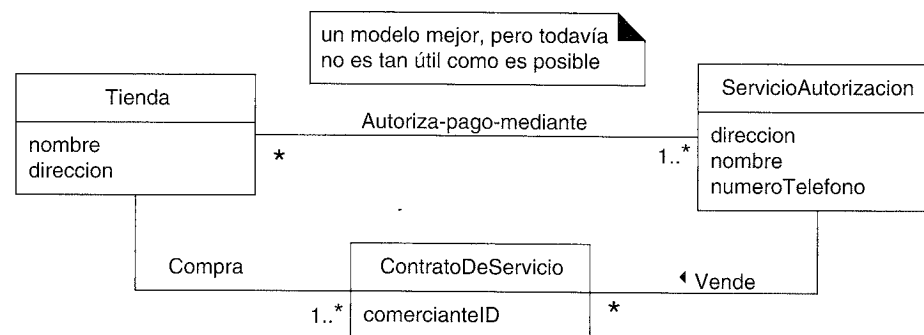


Figura 27.2. Primer intento de modelado del problema del comercianteID.

El hecho de que tanto *Tienda* como *ServicioAutorizacion* estén relacionados con *ContratoDeServicio* es un síntoma de que depende de la relación entre los dos. Se podría pensar en el *comercianteID* como un atributo relacionado con la asociación entre la *Tienda* y el *ServicioAutorizacion*.

Esto nos lleva a la noción de **clase asociación**, en la que podemos añadir características a la propia asociación. Se podría modelar *ContratoDeServicio* como una clase asociación relacionada con la asociación entre *Tienda* y *ServicioAutorizacion*.

En UML, esto se representa con una línea punteada desde la asociación a la clase asociación. La Figura 27.3 transmite visualmente la idea de que un *ContratoDeServicio* y sus atributos están relacionados con la asociación entre una *Tienda* y un *ServicioAutorizacion*, y que el tiempo de vida del *ContratoDeServicio* depende de la relación.

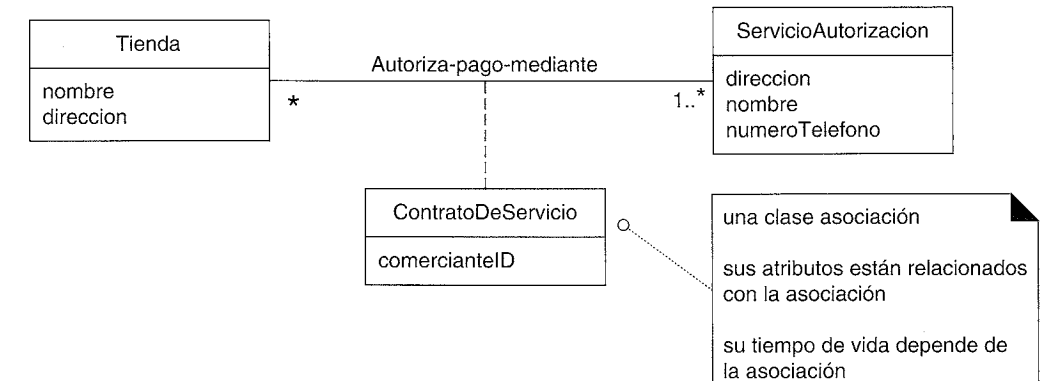


Figura 27.3. Una clase asociación.

Guías

Entre las guías para incluir clases asociaciones se encuentran las siguientes:

Indicios de que podría ser útil una clase asociación en un modelo del dominio:

- Un atributo está relacionado con una asociación.
- El tiempo de vida de las instancias de la clase asociación depende de la asociación.
- Existe una asociación muchos-a-muchos entre dos conceptos, e información asociada con la propia asociación.

La presencia de una asociación muchos-a-muchos es un signo típico de que una clase asociación útil está escondida en segundo plano en algún sitio; cuando vea una, tenga en cuenta una clase asociación.

La Figura 27.4 ilustra algunos otros ejemplos de clases asociación.

27.2. Agregación y composición

La **agregación** es un tipo de asociación que se utiliza para modelar las relaciones todo-parte entre las cosas. El todo se denomina **compuesto**.

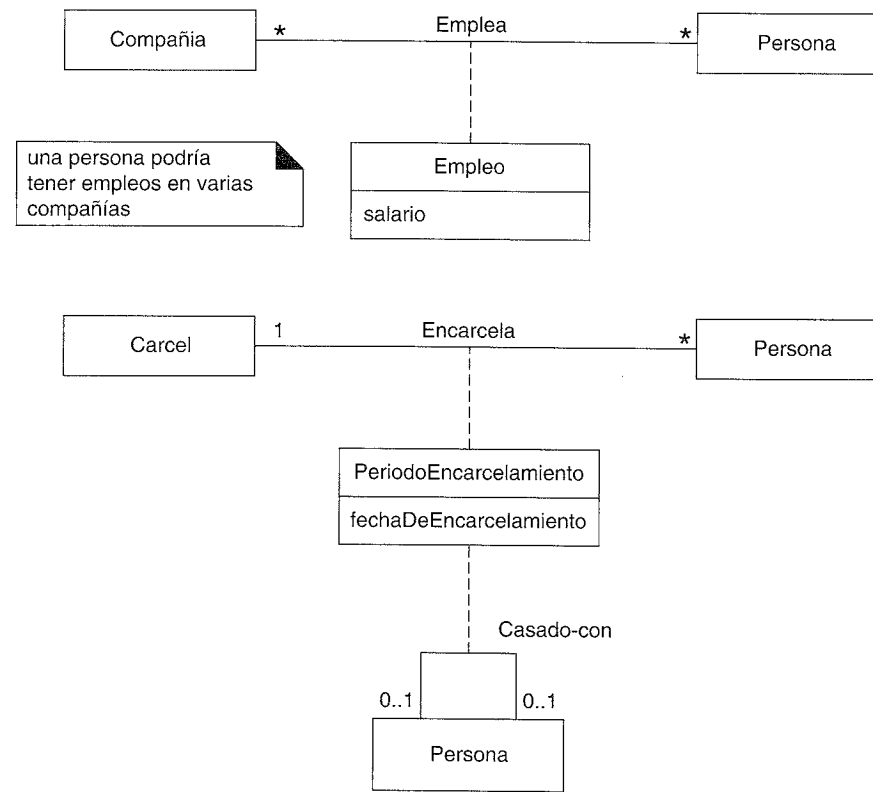


Figura 27.4. Clases asociación.

Por ejemplo, los ensamblajes físicos se organizan mediante relaciones de agregación, del mismo modo que una *Mano* agrega *Dedos*.

Agregación en UML

La agregación se representa en UML mediante un rombo hueco o relleno en el extremo del compuesto de una asociación todo-parte (ver Figura 27.5).

La agregación es una propiedad de un rol de asociación¹.

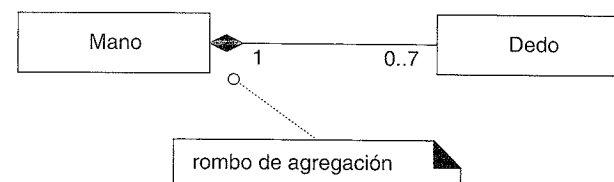


Figura 27.5. Notación de la agregación.

¹ Recordemos que cada extremo de asociación es un rol, y que un rol en UML tiene varias propiedades, como *multiplicidad*, *nombre*, *navegabilidad* y *esAgregado*.

El nombre de la asociación se excluye a menudo de las relaciones de agregación puesto que se piensa habitualmente como *Tiene-parte*. Sin embargo, uno puede acostumbrarse a proporcionar más detalles semánticos.

Agregación de composición: rombo relleno

La **agregación de composición**, o **composición**, significa que la parte es un miembro de un único objeto compuesto y que existe una dependencia de existencia y disposición de la parte sobre el compuesto. Por ejemplo, existe una relación de composición entre la mano y un dedo.

En el Modelo de Diseño, la composición y su implicación de dependencia de existencia indica que los objetos software compuestos crean (o provocan la creación de) los objetos software de la parte (por ejemplo, la *Venta* crea los objetos *LineaDeVenta*).

Pero en el Modelo del Dominio, puesto que no representa objetos software, rara vez es relevante la noción del todo que crea las partes (una venta real no crea una línea de venta real). Sin embargo, existe todavía una analogía. Por ejemplo, en el modelo del dominio de un “cuerpo humano”, uno piensa en la mano que incluye los dedos, luego si uno dice, “Se ha formado una mano”, entendemos que también significa que se han formado los dedos igualmente.

La composición se denota con un rombo relleno. Esto implica que solamente el compuesto posee la parte, y que se encuentran en una jerarquía de partes en forma de árbol; es la forma de agregación más común que se presenta al modelar.

Por ejemplo, un dedo es una parte de una única mano (¡esperamos!), por tanto el rombo de agregación está relleno para indicar agregación de composición (ver Figura 27.6).

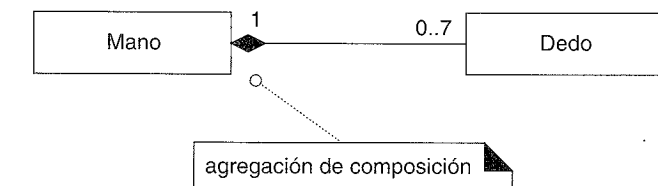


Figura 27.6. Agregación de composición.

Si la multiplicidad en el extremo del compuesto es exactamente uno, la parte *no* podría existir separada de algún compuesto. Por ejemplo, si se quita un dedo de una mano, debe conectarse inmediatamente a otro objeto compuesto (otra mano, pie,...); al menos, eso es lo que está declarando el modelo, ¡independientemente de los méritos métricos de esta idea!

Si la multiplicidad en el extremo del compuesto es 0..1, entonces podría eliminarse la parte del compuesto, y existir todavía sin pertenecer a ningún otro compuesto. Luego, si quiere que los dedos floten en el aire por sí mismos, utilice 0..1.

Agregación compartida: rombo hueco

La **agregación compartida** significa que la multiplicidad en el extremo del compuesto podría ser más de uno, y se representa mediante un rombo hueco. Implica que la parte podría estar simultáneamente en muchas instancias del compuesto. La agregación compartida rara vez (si existe alguna) existe en agregaciones físicas, sino más bien en conceptos no físicos.

Por ejemplo, se podría considerar que un paquete UML agrega sus elementos. Pero un elemento podría ser referenciado en más de un paquete (pertenecer a un paquete, y es referenciado en otros), lo cual es un ejemplo de agregación compartida (ver Figura 27.7).

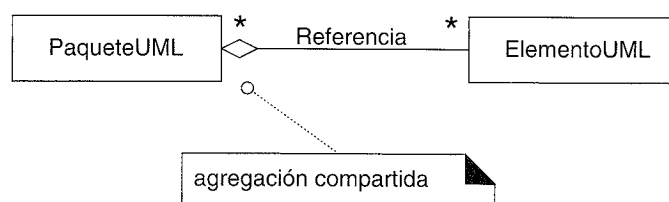


Figura 27.7. Agregación compartida.

Cómo identificar la agregación

En algunos casos, la presencia de la agregación es obvia —normalmente en ensamblajes físicos—. Pero algunas veces, no está claro.

En la agregación: si hay duda, descártela.

A continuación presentamos algunas guías que sugieren cuando mostrar una agregación:

Considere mostrar una agregación cuando:

- El tiempo de vida de la parte está ligado al tiempo de vida del compuesto —existe una dependencia de creación-eliminación de la parte en el todo—.
- Existe un ensamblaje obvio todo-parte físico o lógico.
- alguna propiedad del compuesto se propaga a las partes, como la ubicación.
- Las operaciones que se aplican sobre el compuesto se propagan a las partes, como la destrucción, movimiento o grabación.

Aparte de lo que es un ensamblaje de partes obvio, la siguiente pista más útil es la presencia de una dependencia de creación-eliminación de la parte en el todo.

Un beneficio de la representación de la agregación

Identificar y representar la agregación *no* es excesivamente importante; es bastante posible que se excluya del modelo del dominio. La mayoría —si no todos— los modeladores del dominio con experiencia han visto que se malgasta tiempo improductivo debatiendo los puntos más sutiles de estas asociaciones.

Descubra y muestre las agregaciones si eso le proporciona los siguientes beneficios, la mayoría de ellos se relacionan con el diseño en lugar de con el análisis, que es por lo que no es muy significativa su exclusión del modelo del dominio.

- Aclara las restricciones del dominio en cuanto a la existencia que se desea de la parte independiente del todo. En la agregación de composición, la parte no podría existir fuera del tiempo de vida del todo.
- Durante el trabajo de diseño, esto influye en las dependencias de creación-eliminación entre las clases software del todo y la parte y los elementos de la base de datos (en cuanto a la integridad referencial y los caminos de eliminación en cascada).
- Ayuda en la identificación de un creador (el compuesto) utilizando el patrón GRASP Creador.
- Las operaciones —como la copia y la eliminación— que se aplican al todo a menudo se propagan a las partes.

Agregación en el Modelo del Dominio del PDV

En el dominio del PDV, se podría considerar las instancias *LineaDeVenta* como parte de una *Venta* compuesta; en general, las líneas de una transacción se ven como parte de una transacción agregada (ver Figura 27.8). Además de ajustarse a ese patrón, existe una dependencia de creación-eliminación de las líneas de venta con la *Venta* —sus tiempos de vida están ligados al tiempo de vida de la *Venta*—.

Por una justificación parecida, el *CatalogoDeProductos* es un agregado de objetos *EspecificacionDelProducto*.

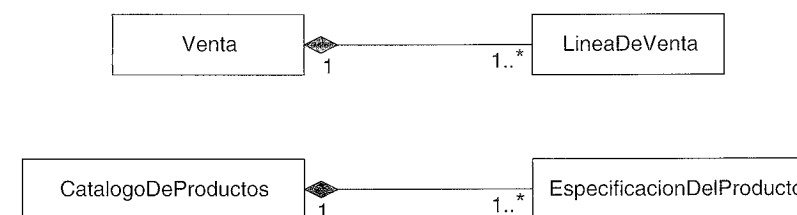


Figura 27.8. Agregación en la aplicación de punto de venta.

Ninguna otra relación es una combinación convincente que sugiera la semántica todo-parte, la dependencia creación-eliminación, y “En caso de duda, descártela”.